UNIVERSITÀ
DI TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer, Communications and Electronic Engineering

Final Dissertation

Energy-Aware Trajectory Optimization of Tracked Vehicles

Supervisor
prof. Michele Focchi

Student
Alessandro Chen

Academic year 2025/2026

“If we don’t build it, they can’t come.”

— Jensen Huang, Founder and CEO of NVIDIA

Acknowledgements

First of all, I would like to thank my parents for their hard work and constant support throughout these years. They have always provided me with the stability, care, and financial support that allowed me to focus on my studies and pursue my interests with greater freedom. Without them, I would certainly have faced much greater pressure and many more difficulties in my daily life. I am deeply grateful for everything they have done for me.

I am also profoundly thankful to the Lord Jesus Christ and for having had the opportunity to grow within my Church. I feel blessed for the life I have been given and for the possibility of serving the Lord. Growing in the House of the Lord has helped me reflect on my weaknesses, my mistakes, and the qualities that I still need to develop. Through my faith, I have learned the importance of humility, clarity of spirit, self-reflection, and the continuous effort to become a better person. It has given me a moral reference that has guided many of my choices and has helped me better understand both myself and the world around me.

I would then like to thank my friend Leonardo Colli, whom I consider a brother. I have known him since my very first day at university, and during these years I have learned a great deal from our friendship. Spending time with him helped me better understand other people, organize my time, develop a stronger work ethic, and study in a more effective and strategic way.

During my first year at university, I often approached subjects according to my curiosity rather than according to what was necessary for the exams. I would spend a great amount of time reconstructing proofs, revisiting theorems and lemmas, and exploring mathematical concepts in depth, while sometimes failing to identify the essential material required to perform well in an examination. Over time, and partly thanks to Leonardo, I learned to distinguish between studying for intellectual curiosity and studying strategically toward a concrete objective. This was an important lesson for me, particularly after finding myself considerably behind with my exams during the first years of my degree.

Without his friendship and influence, I believe it would have been much more difficult for me to adapt, recover academically, and eventually complete my degree on time. He also helped me become more open toward other people and more attentive to their personalities and experiences. Our many conversations about philosophy, psychology, human behaviour, and life have been particularly meaningful to me.

One of the most important things I learned during these years is that life is not simply a competition and that happiness cannot be measured only by achievements or by surpassing others. Friendship, generosity, genuine curiosity, meaningful relationships, and the ability to make the people around us feel respected and comfortable are equally important. My faith has reinforced this understanding: arrogance, excessive ego, hostility, and selfishness create distance not only between people, but also between ourselves and God.

A verse that particularly reminds me of this principle is the commandment to love one's neighbour as oneself.

More generally, I would like to thank all the people I have met during these years and with whom I have shared meaningful moments. Every friendship and every conversation has allowed me to discover perspectives and experiences beyond university. Meeting people with different interests, personalities, ambitions, and ways of living has opened my mind and inspired me to pursue activities and experiences that have made my life richer and more varied.

Finally, I would like to thank the University of Trento for the opportunities it has given me. University has not only provided me with technical and academic knowledge, but has also allowed me to meet many remarkable people and to develop as both a student and a person. It has strengthened my ability to learn quickly, explore concepts deeply, reason critically, and work under significant time pressure. The many challenging examinations and projects I encountered throughout these years often pushed me beyond what I believed I could do, and overcoming these difficulties made me stronger, both intellectually and personally.

These three years have been challenging, sometimes exhausting, but ultimately deeply rewarding. Looking back, I am grateful for the difficulties as much as for the successes, because both contributed to who I am today.

As I conclude this chapter of my life, I hope to continue growing in maturity, discipline, knowledge, and character. Above all, I hope to remain faithful to God, to serve Him more sincerely, and to continue learning from the people and experiences that life will place before me.

To anyone reading these words, I wish the courage to continue moving forward, to remain curious, and to discover the people, experiences, and purposes that can make life meaningful.

Declaration on the Use of Artificial Intelligence Tools

Artificial-intelligence tools, including ChatGPT and Codex, were used as supporting tools during the development and preparation of this thesis. Their use included linguistic revision and improvement of the manuscript, restructuring of technical explanations, assistance with software implementation and debugging, exploration and organization of relevant literature, generation and refinement of experimental ideas, and support in the preparation of data-processing, plotting, and documentation workflows.

These tools were used to support, rather than replace, the author's scientific and engineering work. Experimental objectives, methodological choices, validation procedures, interpretation of results, and final conclusions were reviewed and determined by the author. Suggestions produced by AI tools were verified against the implemented code, experimental outputs, source literature, and the physical assumptions of the project before being incorporated into the thesis.

The author retains full responsibility for the accuracy, originality, and final content of the thesis.

Abstract

Tracked vehicles are well suited for locomotion over soft, uneven, and unstructured terrain, but their skid-steering motion can generate significant relative motion between the tracks and the ground. This produces slip and energy dissipation, so a trajectory that is geometrically short is not necessarily the most energy-efficient one. This thesis investigates how physically meaningful energy costs can be incorporated into trajectory optimization for tracked robots while limiting the computational cost of high-fidelity simulation.

The work builds on a pre-existing high-fidelity terramechanics simulator developed at the University of Trento, which is used throughout the thesis as the physical reference model. The simulator represents both terrain geometry and material properties and models the distributed interaction between the tracks and the ground through multiple contact patches. Although this model does not reproduce every physical phenomenon of the real world, it provides the highest-fidelity description of track–terrain interaction considered in this work.

The first stage of the thesis uses this simulator inside a trajectory-optimization workflow. Several cost formulations are progressively investigated, starting from mechanical energy and later including smoothness, tracking, goal-reaching, slip, and trajectory-duration terms. Since the simulator behaves as an expensive black-box cost function, centered finite differences are adopted to estimate gradients and are integrated into a CHOMP-based optimization framework. This makes physically grounded trajectory deformation possible, but it also requires many independent physical simulation rollouts for every optimization iteration.

To reduce this computational burden, the workflow is parallelized on the UniTrento HPC infrastructure. Distributed CPU resources execute simulator evaluations through PBS resource allocation and a controller–worker architecture with persistent simulation workers. The same infrastructure is then reused to generate larger simulator-labelled datasets and to train machine-learning surrogate models.

The second part of the thesis explores lower-cost approximations of the simulator. These include local power models, simplified physics-based estimates corrected through learned residuals, direct executed-trajectory predictors, and local forward models that recursively predict vehicle motion. The dataset design evolves from early proof-of-concept data to larger V3 and V4 procedural datasets with richer terrain geometry, spatially varying material properties, diverse trajectories, and variable execution durations.

Overall, the thesis develops a pipeline in which high-fidelity simulation acts as a physical teacher, distributed computation makes large-scale simulation practical, and learned models are evaluated as possible lower-cost components for future energy-aware planning. The experiments show that simulator-backed CHOMP can produce large and physically interpretable trajectory deformations, but also that strong supervised prediction metrics do not by themselves guarantee a surrogate model useful for optimization. Preserving the local ordering and cost differences between nearby trajectory perturbations remains the central challenge for reliable surrogate-based CHOMP.

Contents

1 Introduction

1.1 Motivation

Tracked robots are particularly well suited to locomotion over soft, uneven, steep, and unstructured terrain. Their large contact surface distributes the vehicle weight over a wider region than conventional wheels and can provide useful traction on loose soil, agricultural terrain, construction sites, natural landscapes, and disaster-response environments.

The same mechanism that makes tracked locomotion effective also makes it difficult to model and plan. A tracked vehicle normally changes direction by driving its left and right tracks at different speeds. Unlike an ideal differential-drive vehicle, the tracks cannot satisfy pure rolling everywhere along their extended contact surface during a turn. Some relative motion between the tracks and the terrain is therefore intrinsic to skid-steered locomotion.

This observation has an important consequence for trajectory planning: the geometrically shortest path is not necessarily the physically best one.

Consider, for example, a vehicle that must reach the top of a mountain. The shortest geometric route might be an almost straight line toward the summit. Such a path could nevertheless cross steep slopes, loose material, or regions requiring aggressive steering. A longer trajectory following a gentler route may require less mechanical effort, generate less slippage, and ultimately be more reliably executable.

The same distinction applies to tracked robots even when two candidate paths have similar geometric length. Their physical behaviour can differ because of terrain inclination, path curvature, terrain material, vehicle velocity, load distribution, and the amount of relative track–terrain motion produced during execution.

The planning problem considered in this thesis is therefore broader than shortest-path planning. The underlying question is

> Given a tracked robot, a terrain environment, and a start and goal condition, which feasible trajectory allows the robot to reach its destination while reducing undesirable physical quantities such as mechanical energy consumption, slippage, poor tracking, and excessive execution time?

Answering this question requires more than geometric information. A planner must be able to estimate how the robot will actually interact with the terrain.

This introduces the central tension of the thesis:

$$\boxed{\text{physical fidelity} \quad \longleftrightarrow \quad \text{computational efficiency}} \tag{1.1}$$

A richer physical model can provide more meaningful information for planning, but repeatedly evaluating such a model can become prohibitively expensive.

1.2 Research Problem and Gap

This work builds on a pre-existing tracked-vehicle modelling, control, and planning framework developed in the robotics research activity at the University of Trento, in particular the terramechanics and pseudo-kinematic framework presented by Focchi *et al.* [3].

The high-fidelity simulator used throughout this thesis is therefore *not* claimed as a contribution of this work. Instead, it provides the physical reference model on top of which the trajectory-optimization, parallel-computing, dataset-generation, and learning components are developed.

Unlike a purely geometric or ideal kinematic model, the simulator represents the interaction between the extended track contact region and the terrain. Local relative track–terrain motion generates terrain-dependent forces whose combined effect determines the resulting vehicle motion. Consequently, a candidate trajectory can be evaluated not only according to its geometric shape, but also according to quantities related to execution, mechanical effort, and slippage.

In conceptual form,

$$\begin{aligned} \text{desired trajectory} &\rightarrow \text{track–terrain interaction} \\ &\rightarrow \text{forces and vehicle motion} \\ &\rightarrow \text{physical trajectory cost.} \end{aligned} \tag{1.2}$$

The detailed physical formulation is reviewed in Chapter 2.

The difficulty is computational. Evaluating one candidate trajectory requires a complete physical rollout. If this evaluation is embedded inside an iterative trajectory optimizer, the simulator must be executed repeatedly for many slightly different trajectories.

This creates a gap between two desirable properties:

- **high physical fidelity**, which makes the planning objective more representative of the actual tracked-vehicle behaviour;
- **low evaluation cost**, which is required when many candidate trajectories must be compared during optimization.

Previous tracked-vehicle work provides different points along this spectrum, from geometric and pseudo-kinematic planning to more detailed terramechanics models. Gradient-based trajectory optimization methods such as CHOMP [9], on the other hand, provide a mechanism for deforming an entire trajectory according to a local cost gradient.

The combination is attractive but computationally challenging: a trajectory optimizer may require many evaluations of a model that is itself expensive.

This thesis investigates that interface.

1.3 Problem Formulation

A desired planar trajectory is represented by a sequence of N knots,

$$\boldsymbol{\xi} = \{\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_{N-1}\}, \qquad \mathbf{p}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}. \tag{1.3}$$

The initial and final knots define the boundary conditions, while the intermediate knots describe the geometry that may be modified by the planner.

If execution time is also considered, a trajectory can be associated with a duration T. For uniformly spaced knots,

$$\Delta t_{\text{knot}} = \frac{T}{N-1}. \tag{1.4}$$

Changing the spatial knots changes *where* the vehicle is requested to travel. Changing T changes *how quickly* that geometry is traversed.

A general representation of the terrain-aware planning problem is therefore

$$(\boldsymbol{\xi}^\star, T^\star) = \arg\min_{\boldsymbol{\xi}, T} J(\boldsymbol{\xi}, T, \mathcal{T}), \tag{1.5}$$

where $\mathcal{T}$ denotes the terrain environment and J represents a planning objective constructed from the physical and geometric quantities relevant to the considered experiment.

Depending on the formulation, these quantities can include mechanical energy, slip-related cost, smoothness, trajectory-tracking behaviour, goal-reaching accuracy, and execution duration.

The exact objective functions, trajectory parameterization, numerical gradient computation, and optimization procedure are introduced in Chapter 3. The purpose of Equation (1.5) is only to make the central planning problem explicit: both the geometry of the trajectory and, in the more general formulation, its timing can influence its physical cost.

1.4 Thesis Approach

The thesis approaches the fidelity–computation problem through two complementary directions.

1.4.1 High-Fidelity Simulator-Backed Optimization

The first direction retains the high-fidelity simulator directly inside the planning loop.

The simulator is treated as an expensive physical evaluator,

$$\xi \longrightarrow \text{physical rollout} \longrightarrow J(\xi). \tag{1.6}$$

The resulting physical cost is incorporated into a CHOMP-based local trajectory-optimization framework. Since the simulator does not directly provide the required derivatives of the trajectory cost, the local sensitivity of the objective is estimated numerically by evaluating perturbed trajectories.

This immediately creates a large number of independent simulator evaluations. Their independence is exploited through parallel execution on the UniTrento HPC infrastructure.

The role of parallelization should be distinguished carefully from model reduction. Parallel execution can reduce the wall-clock time required to obtain a collection of physical evaluations, but it does not reduce the underlying amount of high-fidelity computation:

$$\begin{aligned} \text{parallelization} &\rightarrow \text{lower latency}, \\ \text{but not} \quad &\rightarrow \text{fewer physical rollouts}. \end{aligned} \tag{1.7}$$

The exact optimization branch therefore investigates how far high-fidelity simulation can be pushed through parallel computing while preserving the original physical model.

1.4.2 Simulator-Trained Surrogate Models

The second direction addresses the computational problem more fundamentally.

Instead of evaluating the complete simulator every time a physical prediction is required, the simulator is used as a *teacher*. Large collections of terrain-conditioned trajectories are executed through the physical model, and the resulting motion and energetic quantities are stored as training labels.

The corresponding workflow is

$$\begin{aligned} \text{high-fidelity simulator} &\rightarrow \text{teacher dataset} \\ &\rightarrow \text{surrogate model} \\ &\rightarrow \text{low-cost physical prediction}. \end{aligned} \tag{1.8}$$

Several levels of approximation are considered.

Direct planner models. These models map trajectory and terrain information directly to planning-relevant outputs, such as trajectory-level energetic quantities or predicted executed motion.

Local power models. These models estimate physical power at local time intervals. Energy is then reconstructed by integration, preserving information about where along the trajectory physical effort is generated.

Local forward models. These models predict the evolution of the executed robot state from local state, command, and terrain information. Recursive rollout can therefore approximate the actual motion generated by the teacher simulator.

Hybrid physics–learning models. Reduced physical approximations can also be retained explicitly, while a learned residual predicts the discrepancy between the inexpensive model and the high-fidelity teacher.

These model families represent different compromises between computational cost, physical interpretability, and the amount of information available to a planner.

A central question, however, is whether conventional predictive accuracy is enough.

A surrogate may achieve low mean error on held-out examples while still providing an inaccurate variation of cost around a candidate trajectory. For gradient-based planning, this distinction is critical. The optimizer does not only require the value of the cost; it also depends on how that value changes when the trajectory is perturbed.

The learning problem considered in this thesis is therefore ultimately not

Can the simulator outputs be predicted accurately?

but rather

Can the simulator be approximated accurately enough that the resulting model remains useful for physical trajectory evaluation and optimization?

1.5 Research Questions

The work is organized around four main research questions.

1. **Physical trajectory optimization.**

   Can a high-fidelity tracked-vehicle simulator provide a sufficiently informative physical objective to guide local trajectory optimization?

2. **Computational scalability.**

   Can the many independent physical evaluations required by simulator-backed trajectory optimization be distributed efficiently enough to make high-fidelity optimization practical for systematic experimentation?

3. **Surrogate modelling and generalization.**

   Can lower-cost learned or hybrid models reproduce planning-relevant energetic and dynamical quantities generated by the simulator, including on terrain environments not used during training?

4. **Planning usefulness of learned models.**

   Does good supervised prediction accuracy imply that a surrogate also reproduces the local physical cost structure required for trajectory optimization?

These questions create a progression from direct physical evaluation to scalable computation and finally to learned approximations. Importantly, the last question evaluates surrogate models according to the purpose for which they are ultimately intended rather than according to regression metrics alone.

1.6 Scope and Contributions

The thesis does not introduce the underlying high-fidelity terramechanics simulator, nor does it claim CHOMP as a new optimization algorithm. Instead, its contribution lies in connecting high-fidelity tracked-vehicle physics with trajectory optimization and studying how the resulting computational burden can be addressed through parallel execution and learned approximations.

The main contributions are grouped into three areas.

1.6.1 High-Fidelity Physical Trajectory Optimization

A simulator-backed trajectory-optimization workflow is developed in which physical quantities produced by the tracked-vehicle simulator are incorporated into a CHOMP-based local optimization procedure.

Because the physical simulator acts as a black-box evaluator from the perspective of the optimizer, the required trajectory sensitivities are estimated numerically. This enables mechanical and slip-related quantities to influence trajectory deformation without replacing the underlying physical model with an analytical approximation.

1.6.2 Scalable Physical Evaluation

The independent physical evaluations required by trajectory optimization are distributed over the UniTrento HPC infrastructure through an application-level controller–worker architecture.

Persistent and isolated simulator workers reduce repeated initialization costs and support both trajectory optimization and large-scale teacher-data generation. The implementation also introduces experiment-management mechanisms for reproducibility and recovery of long-running optimization experiments.

1.6.3 Surrogate Models for Physical Planning

Multiple lower-cost approximations of the high-fidelity simulator are investigated, including direct trajectory-level models, local power models, reduced-physics residual models, and local forward models.

The corresponding datasets are progressively designed to represent planning-scale terrain and trajectory variation, including changes in terrain geometry, material properties, trajectory geometry, local motion, and execution timing.

The learned models are evaluated not only using conventional regression metrics, but also according to physically and geometrically meaningful criteria. These include energetic prediction, executed-trajectory prediction, generalization to unseen terrain, computational speed, and the ability to preserve planning-relevant trajectory and cost structure.

A central experimental finding of the thesis is that strong supervised prediction performance is necessary but not sufficient for surrogate-based trajectory optimization. A model intended to replace the physical simulator inside a local planner must also reproduce how the physical objective varies under trajectory perturbations.

1.7 Thesis Organization

The remainder of the thesis is organized as follows.

Chapter 2 introduces the physical and algorithmic foundations required by the work. It reviews tracked-vehicle terramechanics, the pseudo-kinematic representation, geometric and slippage-aware planning, trajectory optimization with CHOMP, predictive-control approaches such as MPC and MPPI, and learned or hybrid surrogate models.

Chapter 3 presents the methods and implementation developed in this thesis. It defines the trajectory representation and physical objectives, describes the simulator-backed CHOMP formulation and numerical gradient estimation, presents the parallel HPC architecture, and introduces the dataset-generation pipeline, surrogate-model families, and evaluation methodology.

Chapter 4 reports the experimental results. High-fidelity optimization is evaluated through both scalar convergence quantities and trajectory geometry, while the surrogate models are assessed using teacher-versus-model physical predictions, desired-versus-executed trajectory comparisons, unseen-terrain generalization, inference performance, and planning-oriented tests.

Finally, Chapter 5 summarizes the main findings, discusses the limitations of both high-fidelity and surrogate-based optimization, and outlines possible extensions toward more efficient terrain-aware planning for tracked robots.

2 Background and Related Work

This chapter introduces the physical and algorithmic foundations on which the remainder of the thesis is built. Its purpose is not to provide an exhaustive survey of mobile-robot planning or terramechanics. Instead, it develops the chain of ideas that is necessary to understand the problem addressed in this work.

Two questions are particularly important. The first is physical:

> What makes the motion of a tracked vehicle over uneven terrain different from the motion predicted by an ideal geometric or kinematic model?

The second is computational:

> If a detailed physical model can answer the first question, can that model be evaluated cheaply enough to be used repeatedly inside trajectory optimization?

The tracked-vehicle modelling framework developed by Focchi *et al.* [3] provides the main physical foundation for answering the first question. Their work includes a distributed terramechanics simulator, a lower-complexity pseudo-kinematic model, trajectory control, and motion-planning approaches with different accuracy–computation trade-offs.

The second major foundation is CHOMP, Covariant Hamiltonian Optimization for Motion Planning [9]. CHOMP provides a mechanism for deforming an entire trajectory according to the gradient of an objective functional. While the original method was developed primarily for smooth collision-free motion, its trajectory-space formulation makes it attractive for more general physical objectives.

These two lines of work meet at the central difficulty investigated in this thesis. A physically meaningful trajectory cost can be evaluated using a detailed simulator, but such an evaluation is considerably more expensive than evaluating a geometric obstacle or smoothness cost. This motivates both the parallel high-fidelity optimization approach and the lower-cost surrogate models developed later.

The final part of the chapter therefore considers predictive control and learned models. Model Predictive Control (MPC) and Model Predictive Path Integral control (MPPI) illustrate another family of methods in which a model must be evaluated repeatedly over candidate future motions. Learned dynamics and hybrid physics–learning models then provide a natural way of replacing part of this repeated computation with an approximation learned from data.

2.1 Tracked-Vehicle Motion and Terramechanics

Before discussing trajectory optimization, it is useful to understand what a trajectory physically means for a tracked vehicle. A path is not executed in empty geometric space: motion results from forces exchanged between the two tracks and the terrain, and those forces depend on the state of the vehicle, the commanded track motion, the local geometry of the ground, and the mechanical properties of the terrain.

2.1.1 Reference Frames, State, and Vehicle Motion

For planar motion, the vehicle pose can be represented as

$$\boldsymbol{\eta} = \begin{bmatrix} x & y & \phi \end{bmatrix}^\top, \tag{2.1}$$

where x and y describe the Cartesian position of the robot and ϕ is its heading.

Two reference frames appear repeatedly throughout this thesis.

The *world frame* $\mathcal{W}$ is fixed with respect to the terrain. Global positions, the terrain map, and the desired trajectory are naturally expressed in this frame.

The *body frame* $\mathcal{B}$ moves and rotates with the vehicle. Its x_b axis points approximately along the forward direction of the robot, whereas y_b is lateral. Local velocities and track–terrain interaction quantities are conveniently expressed in this frame.

Figure 2.1: Top view of a differentially steered tracked vehicle and its unicycle approximation, including the main reference frames, velocities, forces, and instantaneous centre of rotation. Adapted from Focchi *et al.* [3].

Figure 2.2: Discretization of one track and shear-velocity field during a turning manoeuvre. The different vectors illustrate that track–terrain relative motion varies along the contact surface. Adapted from Focchi *et al.* [3].

The translational velocity expressed in the body frame is

$$ {}^{b}\mathbf{v} = \begin{bmatrix} {}^{b}v_x \\ {}^{b}v_y \end{bmatrix}, \tag{2.2} $$

where ${}^{b}v_x$ is the longitudinal velocity and ${}^{b}v_y$ is the lateral velocity. Rotation about the vertical axis is described by the yaw rate ω_z.

The corresponding world-frame motion is obtained by rotating the local velocity according to the current heading,

$$ \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \mathbf{R}(\phi) \begin{bmatrix} {}^{b}v_x \\ {}^{b}v_y \end{bmatrix}, \qquad \dot{\phi} = \omega_z. \tag{2.3} $$

This distinction is simple but important. A desired trajectory is normally specified globally, whereas the physical interaction with the ground is largely determined by quantities measured relative to the robot itself. Later, the same distinction reappears in the learning problem: the desired path may be represented in world coordinates, while terrain-conditioned dynamics are often easier to learn using local quantities.

Kinematics describe how the vehicle moves. Dynamics explain why that motion occurs. In a body-fixed frame, the translational and rotational Newton–Euler equations can be written in the general form

$$ m\left({}^{b}\dot{\mathbf{v}} + {}^{b}\boldsymbol{\omega} \times {}^{b}\mathbf{v}\right) = \sum {}^{b}\mathbf{F}, \tag{2.4} $$

and

$$ {}^{b}\mathbf{I}\,{}^{b}\dot{\boldsymbol{\omega}} + {}^{b}\boldsymbol{\omega} \times {}^{b}\mathbf{I}\,{}^{b}\boldsymbol{\omega} = \sum {}^{b}\mathbf{M}. \tag{2.5} $$

The cross-product terms appear because the equations are expressed in a frame that rotates together with the robot. On non-flat terrain, the force balance also includes gravity, terrain-normal reactions, tangential track forces, rolling resistance, and the corresponding moments. The detailed derivation is given by Focchi *et al.*; here, the important point is that the vehicle motion results from the combined contribution of many local terrain interactions rather than from an ideal no-slip kinematic constraint.

2.1.2 Skid Steering and Distributed Contact

A conventional differential-drive model assumes that the wheels roll without lateral slip. A tracked vehicle cannot generally satisfy the same assumption during a turn.

Heading is changed by commanding different velocities to the left and right tracks. Because each track has a finite longitudinal contact region and is laterally rigid, the complete track footprint cannot follow a curved path while simultaneously maintaining pure rolling at every point. Some relative motion between the track and the terrain is therefore intrinsic to skid steering.

This is the physical origin of the lateral and longitudinal slippage that distinguishes tracked locomotion from the ideal unicycle model.

The detailed terramechanics model does not represent each track as one point contact. Instead, the contact footprint is discretized into smaller patches. Each patch occupies a different position relative to the vehicle centre and therefore experiences a different local velocity.

This distributed representation is essential to the physical model. A point near the front of a track, for example, does not experience exactly the same relative velocity as a point near the rear when the vehicle is rotating. Consequently, the local shear stress and force can also differ.

Focchi *et al.* report a 10×4 discretization per track in the reference configuration used for their numerical experiments. The exact discretization used in the experiments of this thesis is an implementation detail and is specified separately where required.

2.1.3 From Shear Velocity to Shear Displacement

Consider a patch at body-frame position

$$\mathbf{r}_p = \begin{bmatrix} x_p \\ y_p \end{bmatrix}. \tag{2.6}$$

Even if the vehicle centre translates with velocity ${}^b\mathbf{v}$, the patch velocity is modified by the rotation of the robot,

$$\mathbf{v}_p = {}^b\mathbf{v} + {}^b\boldsymbol{\omega} \times \mathbf{r}_p. \tag{2.7}$$

The moving track itself introduces another velocity. The physically relevant quantity is therefore the relative motion between the track surface and the ground.

For a point on the left track, the shear velocity used by Focchi *et al.* can be expressed as

$${}^b\mathbf{j}_v = \begin{bmatrix} {}^bj_{v,x} \\ {}^bj_{v,y} \end{bmatrix} = \begin{bmatrix} {}^bv_x - \omega_z y_p - \omega_{w,L} r \\ {}^bv_y + \omega_z x_p \end{bmatrix}, \tag{2.8}$$

where $\omega_{w,L}$ is the angular speed of the left sprocket and r is the sprocket radius. The term $\omega_{w,L} r$ therefore represents the longitudinal surface velocity generated by the track. The corresponding expression for the right track is obtained analogously.

Equation (2.8) is particularly informative. Slip at a patch is not determined by wheel speed alone. It also depends on the vehicle translation, the yaw rate, and the position of the patch along the track.

The terramechanics model uses shear *displacement*, rather than only instantaneous shear velocity, to determine the available shear stress. In the reference formulation, the time associated with the passage of a track element from its initial ground contact to a longitudinal coordinate x_p is

$$t_p = \frac{\ell - x_p}{\omega_{w,L} r}, \tag{2.9}$$

where ℓ is the track semi-length.

The accumulated shear displacement can therefore be interpreted as the relative motion integrated over this contact interval. In the formulation of Focchi *et al.*,

$$j_x = \int_{x_p}^{\ell} \frac{{}^wj_{v,x}}{\omega_{w,L} r}\, dx, \qquad j_y = \int_{x_p}^{\ell} \frac{{}^wj_{v,y}}{\omega_{w,L} r}\, dx, \tag{2.10}$$

with magnitude

$$j = \sqrt{j_x^2 + j_y^2}. \tag{2.11}$$

The distinction between velocity and displacement matters physically. Instantaneous relative motion tells us how rapidly the track is sliding, while the shear displacement measures how much relative motion has developed over the considered contact interval.

2.1.4 From Shear Displacement to Terrain Forces

The next step connects the relative motion of the track to the force that can be transmitted through the terrain.

The brush–bristle terramechanics relationship used in the reference model is

$$\tau = c + \sigma\mu \left(1 - e^{-j/K}\right), \tag{2.12}$$

where τ is the shear-stress magnitude, c is the terrain cohesion, σ is the normal pressure, μ is the friction coefficient, j is the shear displacement, and K is the shear-deformation modulus.

The significance of Equation (2.12) is easier to understand than its notation might initially suggest. A small relative displacement does not necessarily mobilize all the shear force that the terrain can provide. As the relative displacement increases, the available shear stress approaches a terrain-dependent limiting value.

Terrain geometry alone is therefore insufficient to determine the physical cost of motion. Two patches with the same inclination can respond differently if their friction, cohesion, loading, or deformation properties differ.

Under the simplified uniform-load assumption, the normal pressure is

$$\sigma_{ij} = \frac{mg}{2A_t}, \tag{2.13}$$

where A_t is the contact area of one track. Notice that this quantity is a *pressure*, not the normal force itself.

In the more detailed non-uniform loading formulation, the terrain reaction is evaluated at individual patches. A local normal force $f_{n,ij}$ produces a corresponding local pressure approximately proportional to

$$\sigma_{ij} \propto \frac{f_{n,ij}}{A_p}, \tag{2.14}$$

where A_p is the patch area. This enables the loading to shift across the tracks as the robot changes orientation on uneven or sloped terrain.

The magnitude of the shear stress is given by Equation (2.12), while its force direction is opposite to the local shear velocity. Introducing the angle δ with respect to the body x_b axis,

$$\sin\delta = \frac{{}^b j_{v,y}}{\|{}^b\mathbf{j}_v\|}, \qquad \cos\delta = \frac{{}^b j_{v,x}}{\|{}^b\mathbf{j}_v\|}, \tag{2.15}$$

the force contribution associated with a small contact area dA is

$${}^b dF_x = -\tau\cos(\delta)\, dA, \tag{2.16}$$

$${}^b dF_y = -\tau\sin(\delta)\, dA. \tag{2.17}$$

Because the patch is generally offset from the robot centre, the same force also produces a moment,

$${}^b dM_z = -y_p\, {}^b dF_x + x_p\, {}^b dF_y. \tag{2.18}$$

The total contribution of one track is obtained by summing over its patches,

$${}^b F_x = \sum_{i=1}^{n} {}^b dF_{x,i}, \qquad {}^b F_y = \sum_{i=1}^{n} {}^b dF_{y,i}, \qquad {}^b M_z = \sum_{i=1}^{n} {}^b dM_{z,i}. \tag{2.19}$$

The same computation is performed for the opposite track. Together with gravity, terrain-normal reactions, rolling resistance, and the remaining dynamic terms, these forces and moments determine the acceleration of the vehicle. Numerical integration then advances the robot state.

The physical chain can therefore be summarized as

$$\begin{aligned} \text{robot state and track motion} &\rightarrow \text{local patch velocity} \\ &\rightarrow \text{shear velocity} \rightarrow \text{shear displacement} \\ &\rightarrow \text{shear stress} \rightarrow \text{patch forces and moments} \\ &\rightarrow \text{total rigid-body forces} \rightarrow \text{next robot state}. \end{aligned} \tag{2.20}$$

This is the important conceptual bridge between trajectory geometry and physical trajectory cost. Moving a trajectory knot changes the commanded motion of the vehicle. That modification changes the states and velocities encountered during execution, which in turn changes the local track–terrain interaction, the required forces, and ultimately quantities such as mechanical power, energy, slip, and tracking error.

2.1.5 Physical Fidelity and Computational Cost

The same distributed formulation that provides richer physical information also explains why high-fidelity simulation is expensive.

At each simulator step, terrain contact must be evaluated; the local motion of multiple patches must be determined; normal and tangential interactions must be computed; patch contributions must be accumulated; the rigid-body equations must be solved; and the resulting state must be numerically integrated. These operations are then repeated over the complete duration of a trajectory.

The difference between uniform and non-uniform loading reported by Focchi *et al.* provides a useful illustration. In their experiments, the average simulation-loop time was approximately 0.49 ms under the uniform loading approximation and approximately 11 ms when the normal interaction was evaluated at the track patches [3].

The absolute numbers depend on the implementation and hardware, but the general lesson is independent of them:

$$\text{greater physical detail} \quad \Longrightarrow \quad \text{greater computational cost.} \tag{2.21}$$

For a single trajectory rollout this cost may be acceptable. Inside an optimizer, however, the same simulator can be called many times to evaluate different candidate trajectories. The computational problem therefore becomes much more severe.

2.2 Reduced Models and Tracked-Vehicle Planning

The cost of detailed terramechanics motivates a natural question: how much of the physical behaviour must be retained in order to plan or control the vehicle effectively?

One answer is to replace the distributed interaction model with a smaller set of variables that summarize its most important consequences.

2.2.1 Pseudo-Kinematic Representation

The classical unicycle model assumes zero lateral velocity. Its planar kinematics are

$$\begin{aligned} \dot{x} &= v\cos\phi, \\ \dot{y} &= v\sin\phi, \\ \dot{\phi} &= \omega_z. \end{aligned} \tag{2.22}$$

This representation is computationally attractive but cannot directly capture the lateral skid motion of a tracked vehicle.

Focchi *et al.* instead introduce a pseudo-kinematic model that preserves a compact kinematic structure while embedding the main effect of terramechanics through three slip quantities: the lateral slip angle α and the longitudinal slips β_L and β_R of the two tracks.

The lateral angle is related to the local velocity components by

$$\alpha = \tan^{-1}\left(\frac{{}^{b}v_y}{{}^{b}v_x}\right). \tag{2.23}$$

The vehicle kinematics then become

$$\dot{x} = \frac{{}^{b}v_x}{\cos\alpha}\cos(\phi+\alpha), \tag{2.24}$$

$$\dot{y} = \frac{{}^{b}v_x}{\cos\alpha}\sin(\phi+\alpha), \tag{2.25}$$

$$\dot{\phi} = \omega_z. \tag{2.26}$$

The longitudinal slips describe the difference between track motion relative to the ground and the velocity imposed by the sprocket. For the left track,

$$\beta_L = {}^{b}v_L^t - \omega_{w,L}r, \tag{2.27}$$

and analogously for the right track.

Instead of resolving dozens of local contact interactions, the reduced model therefore represents their macroscopic effect through a small number of quantities.

This is an important idea for the present thesis because it introduces a general modelling hierarchy:

$$\begin{aligned} \text{geometric/kinematic model} &\longrightarrow \text{pseudo-kinematic model} \\ &\longrightarrow \text{distributed terramechanics model.} \end{aligned} \tag{2.28}$$

Moving from left to right generally provides a richer physical description, but requires greater computational effort.

Earlier work on tracked vehicles similarly estimated simplified slip parameters for odometry, modelling, and control [4, 8]. The contribution of the framework of Focchi *et al.* is particularly relevant here because the reduced model and the distributed physical simulator belong to the same modelling and planning framework used as the basis of this thesis.

2.2.2 From Geometric to Slippage-Aware Planning

The planning comparison of Focchi *et al.* makes the fidelity–computation trade-off particularly clear.

At the geometric end, Dubins curves connect two planar configurations using straight segments and constant-curvature arcs under a bounded-curvature constraint [2]. They are inexpensive because the solution has a compact analytical structure, but the underlying model does not account for tracked-vehicle slippage.

Clothoids provide a smoother alternative by varying curvature continuously. They avoid some of the abrupt curvature transitions of Dubins paths while remaining inexpensive to generate.

At the more physically informed end, Focchi *et al.* formulate a slippage-aware optimal-control problem based on their pseudo-kinematic model. The resulting trajectory is more consistent with the behaviour of the tracked vehicle and can directly incorporate actuation and velocity constraints. However, numerical optimization requires substantially greater computation.

Their experiments therefore illustrate a recurring pattern:

$$\begin{aligned} &\text{simple model} \rightarrow \text{fast planning but larger model mismatch,} \\ &\text{richer model} \rightarrow \text{more informed planning but larger computational cost.} \end{aligned} \tag{2.29}$$

Numerical trajectory optimization also introduces another important property: the solution is generally local. Convergence can depend on the initial trajectory, and a nonlinear solver is not guaranteed to find the globally best route.

This point is directly relevant to the CHOMP experiments presented later. A gradient-based optimizer can improve a trajectory around its initialization, but changing numerical parameters such as the step size does not transform a local optimizer into a global planner.

2.3 Trajectory Optimization and CHOMP

The planners discussed above generate a trajectory according to a particular vehicle or curve model. A different perspective is to treat the trajectory itself as the optimization variable.

Let a discretized trajectory be written as

$$\boldsymbol{\xi} = \left[\mathbf{q}_0, \mathbf{q}_1, \ldots, \mathbf{q}_{N-1}\right]. \tag{2.30}$$

The endpoints may be fixed while the internal knots are moved in order to reduce a cost $J(\boldsymbol{\xi})$.

Conceptually,

$$\text{initial trajectory} \rightarrow \text{cost and gradient} \rightarrow \text{trajectory deformation} \rightarrow \text{new trajectory.} \tag{2.31}$$

Repeated application of this process produces a sequence of locally improved trajectories.

2.3.1 Original CHOMP Objective

CHOMP was introduced as a gradient-based trajectory-optimization method that combines task objectives with a smoothness metric [9]. A representative objective is

$$J(\boldsymbol{\xi}) = \lambda_{\text{smooth}} J_{\text{smooth}}(\boldsymbol{\xi}) + \lambda_{\text{obs}} J_{\text{obs}}(\boldsymbol{\xi}). \tag{2.32}$$

The smoothness term penalizes irregular motion. In a discretized representation it can be written as a quadratic form,

$$J_{\text{smooth}} = \frac{1}{2}\boldsymbol{\xi}^\top \mathbf{A}\boldsymbol{\xi} + \mathbf{b}^\top \boldsymbol{\xi} + c, \tag{2.33}$$

where the structure of $\mathbf{A}$ depends on the finite-difference operator used to penalize velocity, acceleration, or another derivative of the trajectory.

The obstacle term in the original formulation is constructed from a workspace obstacle potential. A distance field provides each workspace location with information about its distance to the nearest obstacle. This can be converted into a potential $c(\mathbf{x})$ that is small in safe regions and increases close to obstacles.

For a simplified point trajectory,

$$J_{\text{obs}} = \int c(\boldsymbol{\xi}(t)) \left\| \dot{\boldsymbol{\xi}}(t) \right\| dt. \tag{2.34}$$

The spatial gradient of this potential gives the optimizer a direction in which the trajectory can move away from high-cost regions.

The idea is important for this thesis even though the physical objective is different. CHOMP does not fundamentally require the task cost to represent an obstacle. It requires a cost whose variation with respect to the trajectory can be estimated.

2.3.2 The Covariant Update and the Elastic-Band Interpretation

A representative discretized CHOMP update is

$$\boldsymbol{\xi}^{(r+1)} = \boldsymbol{\xi}^{(r)} - \eta \mathbf{A}^{-1} \nabla_{\boldsymbol{\xi}} J\left(\boldsymbol{\xi}^{(r)}\right), \tag{2.35}$$

where r denotes the optimization iteration.

Two parts of Equation (2.35) deserve particular attention.

The scalar η is the *step size*. It determines how far the trajectory moves during one update. If η is too small, the optimizer may make very little progress. If it is too large, the update can move beyond the region where the local gradient provides a useful descent direction.

The matrix $\mathbf{A}$ is more subtle. Ordinary Euclidean gradient descent would use

$$\Delta\boldsymbol{\xi} = -\eta \nabla J. \tag{2.36}$$

Such an update can move one trajectory knot strongly while leaving its neighbours almost unchanged, potentially generating an irregular path.

CHOMP instead preconditions the gradient using $\mathbf{A}^{-1}$,

$$\Delta\boldsymbol{\xi} = -\eta \mathbf{A}^{-1} \nabla J. \tag{2.37}$$

Because $\mathbf{A}$ represents the trajectory smoothness metric, the effect of a local cost gradient is propagated through neighbouring trajectory variables.

A useful mental image is an elastic band stretched between fixed endpoints. Pulling one point of the band does not move only that mathematical point; the surrounding region also deforms. The elastic-band analogy is only an intuition, but it captures the role played by the smoothness metric in the update.

This property is particularly useful for the problem considered in this thesis. A physical cost may indicate that one region of a trajectory is energetically undesirable, but the response should remain a coherent trajectory rather than an isolated displacement of one knot.

2.3.3 Local Optimization and Physical Costs

CHOMP is a local trajectory optimizer. The gradient describes how the cost changes around the current trajectory; it does not systematically explore all possible homotopy classes or all distant alternatives.

Consequently,

$$\text{different initialization} \quad \Longrightarrow \quad \text{potentially different local minimum.} \tag{2.38}$$

This distinction is essential when interpreting the experiments. A straight initial trajectory and an S-shaped initial trajectory can converge to different solutions even when start, goal, world, and cost are identical.

The work developed later in this thesis retains the CHOMP trajectory-space optimization mechanism but changes the information supplied by the task cost. Instead of relying only on an analytical obstacle potential, the objective can contain quantities evaluated through tracked-vehicle simulation.

The exact physical objective, numerical gradient construction, line search, and implementation are methodological contributions of this thesis and are therefore introduced in Chapter 3, rather than in the present background chapter.

2.4 Predictive Planning: MPC and MPPI

CHOMP modifies the complete trajectory directly. Model Predictive Control takes a different view: it repeatedly predicts how candidate control inputs will move the system into the future.

This perspective is relevant because it makes the role of a forward model explicit. It also provides a natural bridge to the learned forward models considered later.

2.4.1 Model Predictive Control

Consider a discrete system

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \tag{2.39}$$

where $\mathbf{x}_k$ is the current state, $\mathbf{u}_k$ is a control input, and f predicts the next state.

For a mobile robot, $\mathbf{x}_k$ might contain quantities such as position, heading, translational velocity, and angular velocity. The control $\mathbf{u}_k$ might represent body velocity commands or, for a tracked platform, left and right track commands.

MPC considers a finite prediction horizon of H steps and solves a problem of the form

$$\min_{\mathbf{u}_0,\dots,\mathbf{u}_{H-1}} \left[\sum_{k=0}^{H-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_H) \right], \tag{2.40}$$

subject to

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k). \tag{2.41}$$

The running cost ℓ can penalize tracking error, control effort, terrain difficulty, energy, or other quantities. The terminal cost ℓ_f expresses how desirable the final predicted state is.

A crucial property is that the complete optimized sequence is not executed. Only the first action is applied. The new state is then measured and the optimization is solved again:

$$\text{observe} \rightarrow \text{predict} \rightarrow \text{optimize} \rightarrow \text{execute one action} \rightarrow \text{repeat}. \tag{2.42}$$

This receding-horizon mechanism allows the controller to react to deviations between prediction and execution.

For a tracked vehicle on uneven terrain, the usefulness of MPC therefore depends strongly on f. If the model predicts ideal no-slip motion while the real system experiences substantial lateral drift, an apparently good predicted control sequence can produce poor execution.

2.4.2 Model Predictive Path Integral Control

MPPI belongs to the same broad predictive-control family, but explores the future using many sampled control sequences rather than relying on a conventional gradient-based optimization of the controls [6, 7].

Starting from a nominal sequence $\bar{\mathbf{u}}_k$, sample m uses

$$\mathbf{u}_k^{(m)} = \bar{\mathbf{u}}_k + \boldsymbol{\epsilon}_k^{(m)}, \tag{2.43}$$

where $\boldsymbol{\epsilon}_k^{(m)}$ is a sampled perturbation.

Each candidate is rolled through the forward model,

$$\mathbf{x}_{k+1}^{(m)} = f\left(\mathbf{x}_k^{(m)}, \mathbf{u}_k^{(m)}\right), \tag{2.44}$$

and receives a trajectory cost S_m. Low-cost samples receive larger weights, schematically

$$w_m \propto \exp\left(-\frac{S_m}{\lambda}\right). \tag{2.45}$$

The nominal control sequence is then shifted toward the perturbations that produced better predicted trajectories.

The difference from CHOMP can be summarized intuitively.

CHOMP asks:

> In which local gradient direction should the current trajectory be deformed?

MPPI asks:

> Among many sampled future control sequences, which ones produce better predicted outcomes?

The MPPI literature includes demonstrations on aggressive autonomous driving, including operation on a dirt test track [7]. Such examples are particularly relevant to terrain-aware robotics because they demonstrate the practical value of performing many nonlinear model rollouts in parallel.

MPC and MPPI are not the main planners implemented in this thesis. Their importance here is conceptual: both show how planning quality and computational cost can become dominated by the repeated evaluation of a forward model. If that model can be approximated cheaply without losing the relevant physical behaviour, many more candidate motions can be evaluated.

2.5 Learned and Hybrid Surrogate Models

The previous sections expose the same problem from different directions. Detailed terramechanics provides physically meaningful predictions but is expensive. Trajectory optimization and predictive control may require the model to be evaluated many times. This naturally motivates surrogate models.

A surrogate does not need to reproduce every internal variable of the original simulator. It should reproduce the quantities required by the downstream task with sufficient accuracy.

2.5.1 Learned Forward Dynamics

A learned forward model replaces the analytical mapping in Equation (2.39) with a parameterized function,

$$\hat{\mathbf{x}}_{k+1} = f_\theta\left(\mathbf{x}_k, \mathbf{u}_k, \mathbf{g}_k\right), \tag{2.46}$$

where $\mathbf{g}_k$ represents relevant environmental information such as local terrain geometry and θ denotes the learned parameters.

Learned dynamics have been used in model-based control precisely because a trained network can provide many state-transition predictions without executing the original physical process [5, 1].

For terrain-dependent motion, the additional input $\mathbf{g}_k$ is particularly important. The same commanded motion can result in different executed motion on flat, inclined, rough, or low-friction terrain.

There is also an important distinction between *one-step* and *rollout* prediction.

During one-step evaluation,

$$\mathbf{x}_k \rightarrow f_\theta \rightarrow \hat{\mathbf{x}}_{k+1}, \tag{2.47}$$

the true current state is supplied to the model.

During autoregressive rollout, however,

$$\hat{\mathbf{x}}_{k+2} = f_\theta\left(\hat{\mathbf{x}}_{k+1}, \mathbf{u}_{k+1}, \mathbf{g}_{k+1}\right). \tag{2.48}$$

The prediction from one step becomes the input to the next. Small errors can therefore accumulate over time. A model with excellent one-step RMSE is not necessarily a good trajectory predictor.

For a model intended for planning, complete-rollout geometry is consequently as important as local regression accuracy.

2.5.2 Predicting Power and Energy

Not every planning problem requires the complete robot state to be predicted. If the objective is energy, it may instead be useful to approximate a local physical quantity such as mechanical power.

Power is the rate at which mechanical work is performed. In a rigid-body description, a generic expression is

$$P = \mathbf{F}^\top \mathbf{v} + \mathbf{M}^\top \boldsymbol{\omega}. \tag{2.49}$$

Energy over a trajectory is obtained by integrating power,

$$E = \int_0^T P(t)\, dt, \tag{2.50}$$

or, in discrete form,

$$E \approx \sum_k P_k \Delta t_k. \tag{2.51}$$

This provides a different surrogate strategy: instead of asking a model to predict one scalar for the complete trajectory, it can predict a sequence of local power values from motion and terrain information. The complete energy is then reconstructed by integration.

Local prediction provides greater temporal and spatial resolution. It can show *where* a trajectory becomes expensive, whereas a single trajectory-level scalar reveals only the final accumulated value.

2.5.3 Hybrid Physics and Learned Residuals

Learning does not necessarily require replacing physics completely.

Suppose an inexpensive physical approximation produces

$$y_{\text{physics}}. \tag{2.52}$$

A learned model can estimate only the part missing from that approximation,

$$\hat{y} = y_{\text{physics}} + \Delta y_\theta. \tag{2.53}$$

This is often called a residual or hybrid physics–learning formulation.

The motivation is straightforward. Quantities that are already explained well by simple mechanics do not need to be rediscovered entirely from data. The network can concentrate its capacity on effects that the reduced physical model neglects.

For tracked locomotion, this idea is especially attractive because there is a large modelling gap between a simple rigid-body or inverse-dynamics approximation and the distributed terramechanics simulator.

Several approximation levels are therefore possible in principle:

$$\begin{aligned} \text{desired trajectory and terrain} &\rightarrow \text{trajectory-level physical quantity}, \\ \text{local state and terrain} &\rightarrow \text{local power}, \\ \text{reduced physics and terrain} &\rightarrow \text{learned residual}, \\ \text{current state, command, terrain} &\rightarrow \text{next executed state}. \end{aligned} \tag{2.54}$$

These alternatives answer different questions. A direct scalar model is attractive when only ranking candidate trajectories is required. A local power model provides a physically interpretable time series. A forward model is needed when the difference between desired and actually executed motion is important.

The exact architectures investigated in this thesis, and the way in which these model families are combined, are introduced in Chapter 3.

2.6 Positioning of This Thesis

The literature reviewed in this chapter reveals a continuous spectrum between model simplicity and physical fidelity.

At one end, geometric methods such as Dubins curves generate trajectories very quickly but largely abstract away the track–terrain interaction. Reduced pseudo-kinematic approaches retain the dominant effects of slippage through a small number of interpretable quantities and are therefore much more suitable for real-time control and planning than a distributed terramechanics model.

At the other end, the simulator of Focchi *et al.* resolves the track–terrain interaction at a considerably finer physical level. It can therefore expose quantities that are difficult to obtain from a geometric planner, including local contact forces, slip-related effects, executed motion, and the physical effort required by the vehicle. The price is a much larger computational cost.

CHOMP provides a complementary piece of the problem. It offers a principled way of deforming a trajectory according to a cost gradient, but its original applications rely on task costs whose gradients can be obtained much more cheaply than a complete terramechanics simulation.

Predictive-control methods such as MPC and MPPI expose the same computational difficulty from another direction: planning becomes expensive whenever many future model evaluations are required. Learned dynamics and hybrid surrogate models show that some of these predictions can instead be approximated from data.

The present thesis connects these ideas through two complementary paths.

First, the high-fidelity simulator is retained directly as the physical reference for trajectory optimization. This makes it possible to investigate whether physically meaningful quantities such as mechanical energy and slip-related energetic losses can guide the deformation of a tracked-vehicle trajectory.

Second, the same simulator is treated as a teacher for lower-cost surrogate models. The objective is not merely to reproduce simulator outputs with low regression error, but to determine which representations preserve enough physical information to remain useful for trajectory evaluation and, ultimately, optimization.

The contribution is therefore neither a replacement for the terramechanics model of Focchi *et al.* nor a new derivation of CHOMP. Rather, it studies the interface between these two existing foundations:

$$\begin{aligned} &\text{high-fidelity tracked-vehicle physics} \longrightarrow \text{physical trajectory cost} \\ &\qquad\qquad\qquad\qquad\qquad\qquad \longrightarrow \text{trajectory optimization} \\ &\qquad\qquad\qquad\qquad\qquad\qquad \longrightarrow \text{parallel and learned approximations.} \end{aligned} \tag{2.55}$$

The next chapter specifies how this connection is implemented in practice: the trajectory representation, physical objectives, numerical gradients, parallel execution architecture, simulator-labelled datasets, and surrogate models are introduced in Chapter 3.

3 Method and Implementation

This chapter describes the methodology developed to connect high-fidelity tracked-vehicle simulation with trajectory optimization and, subsequently, with lower-cost surrogate models. The organization follows the logical structure of the final framework rather than the chronological sequence in which individual software components were developed.

The complete methodology consists of four closely connected layers. First, a desired trajectory is represented geometrically and temporally, and the kinematic quantities that can be obtained directly from that representation are computed analytically. Second, the trajectory is executed through the high-fidelity terramechanics simulator, which provides the physically meaningful quantities used as optimization objectives and later as machine- learning labels. Third, the simulator is embedded inside a CHOMP-based local trajectory optimizer, where numerical perturbations are used to estimate the physical gradient. Because this requires a large number of independent rollouts, the evaluations are distributed across the UniTrento HPC infrastructure. Finally, the same simulator and parallel execution framework are reused to generate structured teacher datasets from which lower-cost surrogate models can be trained.

The high-fidelity simulator therefore has two distinct roles throughout the chapter. During exact optimization it acts as the physical cost evaluator,

$$\boldsymbol{\xi} \longrightarrow \text{high-fidelity rollout} \longrightarrow J_{\text{physical}}(\boldsymbol{\xi}), \tag{3.1}$$

whereas during learning it acts as the teacher,

$$(\mathcal{T}, \boldsymbol{\xi}) \longrightarrow \text{high-fidelity rollout} \longrightarrow \text{teacher labels}. \tag{3.2}$$

Keeping these two roles connected is important: the learned models are not trained against an unrelated analytical approximation, but against the same physical reference used by the simulator-backed planner.

3.1 Trajectory Representation and Derived Quantities

A desired planar trajectory is represented by a sequence of N spatial knots,

$$\boldsymbol{\xi} = \{\mathbf{p}_0, \ldots, \mathbf{p}_{N-1}\}, \qquad \mathbf{p}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}. \tag{3.3}$$

The first and final knots are fixed,

$$\mathbf{p}_0 = \mathbf{p}_{\text{start}}, \qquad \mathbf{p}_{N-1} = \mathbf{p}_{\text{goal}}, \tag{3.4}$$

while the internal knots constitute the spatial optimization variables. The main high-fidelity CHOMP experiments use $N = 42$ total knots, corresponding to $N_{\text{int}} = 40$ internal knots.

The displacement and length of trajectory segment k are

$$\Delta\mathbf{p}_k = \mathbf{p}_{k+1} - \mathbf{p}_k, \qquad \Delta s_k = \|\Delta\mathbf{p}_k\|, \tag{3.5}$$

and the total geometric path length is

$$L = \sum_{k=0}^{N-2} \Delta s_k. \tag{3.6}$$

A total execution duration T is associated with the trajectory. For uniformly spaced knot times,

$$\Delta t_{\text{knot}} = \frac{T}{N-1}, \qquad t_k = k\Delta t_{\text{knot}}. \tag{3.7}$$

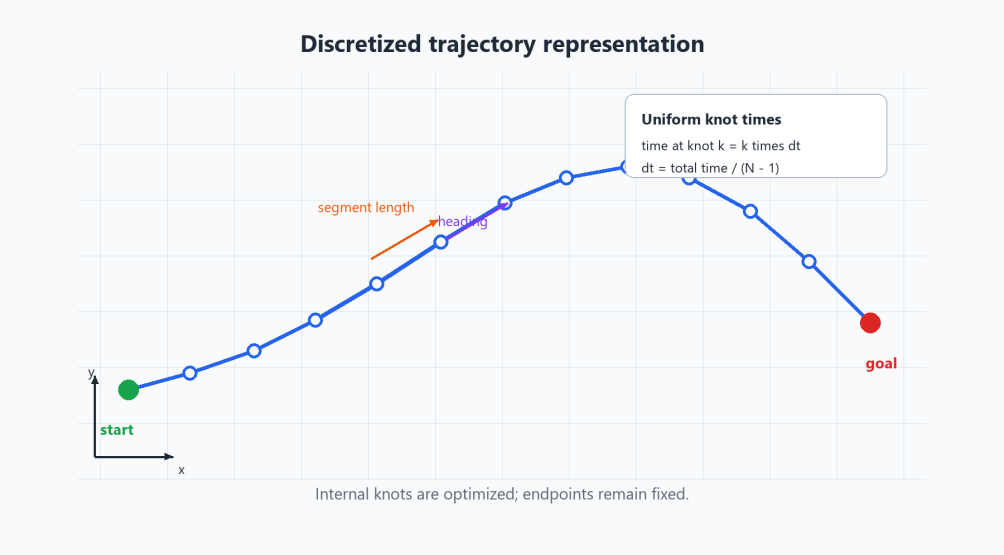


Figure 3.1: Discrete trajectory representation with fixed start and goal, internal optimization knots, segment displacement, heading, and uniform knot timing.

This representation makes an important distinction between geometry and timing. Moving the knots changes *where* the robot is requested to travel, while modifying T changes *how quickly* the same spatial trajectory is executed. The completed high-fidelity CHOMP experiments optimize only the spatial internal knots while keeping T fixed. The later V4 dataset introduces variation in T so that spatiotemporal extensions of the planner can also be studied.

Several useful quantities can be obtained directly from the desired trajectory without executing the simulator. These quantities are treated as preprocessing features rather than teacher labels. The desired heading is

$$\psi_k = \operatorname{atan2}\left(y_{k+1} - y_k, x_{k+1} - x_k\right), \tag{3.8}$$

and the wrapped change in heading is

$$\Delta\psi_k = \operatorname{wrap}\left(\psi_{k+1} - \psi_k\right). \tag{3.9}$$

Assuming the knot timing of Equation (3.7), translational velocity and scalar speed are approximated by

$$^{w}\mathbf{v}_k \approx \frac{\mathbf{p}_{k+1} - \mathbf{p}_k}{\Delta t_{\text{knot}}}, \qquad v_k = \frac{\Delta s_k}{\Delta t_{\text{knot}}}, \tag{3.10}$$

while the desired yaw rate is

$$\omega_{z,k} \approx \frac{\Delta\psi_k}{\Delta t_{\text{knot}}}. \tag{3.11}$$

Translational acceleration is obtained by differentiating the discrete velocity sequence,

$$^{w}\mathbf{a}_k \approx \frac{^{w}\mathbf{v}_{k+1} - {}^{w}\mathbf{v}_k}{\Delta t_{\text{knot}}}, \tag{3.12}$$

and can be expressed in a heading-aligned body frame as

$$^{b}\mathbf{a}_k = \mathbf{R}^{T}(\psi_k)\,{}^{w}\mathbf{a}_k. \tag{3.13}$$

Similarly,

$$\dot{\omega}_{z,k} \approx \frac{\omega_{z,k+1} - \omega_{z,k}}{\Delta t_{\text{knot}}} \tag{3.14}$$

approximates the yaw acceleration, while a discrete estimate of path curvature is

$$\kappa_k \approx \frac{\Delta\psi_k}{\frac{1}{2}\left(\Delta s_{k-1} + \Delta s_k\right)}. \tag{3.15}$$

Finally, the cumulative distance and normalized trajectory progress are

$$s_k = \sum_{i=0}^{k-1} \Delta s_i, \qquad \rho_k = \frac{s_k}{L}. \tag{3.16}$$

These analytically derived quantities describe the motion requested by the desired trajectory. They must be distinguished from the corresponding quantities generated by the simulator during physical execution.

3.2 High-Fidelity Physical Evaluation

A desired trajectory becomes physically meaningful only after it is executed through the tracked-vehicle simulator. The complete evaluation chain can be summarized as

$$\begin{aligned} \text{desired trajectory} &\rightarrow \text{controller and track commands} \\ &\rightarrow \text{track–terrain interaction} \\ &\rightarrow \text{forces and moments} \\ &\rightarrow \text{rigid-body dynamics} \\ &\rightarrow \text{teacher-executed trajectory}. \end{aligned} \tag{3.17}$$

The desired and executed trajectories are intentionally kept distinct. The former is the command supplied to the simulator, whereas the latter is the motion that actually results from the physical interaction between the robot and the terrain. The same distinction is later preserved when introducing surrogate models: desired, teacher-executed, and model-predicted trajectories are never treated as interchangeable signals.

Three time resolutions appear in the implementation,

$$\Delta t_{\text{sim}} \ll \Delta t_{\text{label}} \leq \Delta t_{\text{knot}}, \tag{3.18}$$

with approximately

$$\Delta t_{\text{sim}} \approx 0.001\,\text{s}, \qquad \Delta t_{\text{label}} = 0.05\,\text{s}. \tag{3.19}$$

The simulator timestep is the fine numerical integration interval used by the physical model. The knot interval belongs to the desired trajectory representation, while the label interval determines the temporal resolution at which teacher quantities are stored for learning. Maintaining distinct names for these quantities avoids ambiguity when trajectory timing and label generation use different temporal resolutions.

3.2.1 Mechanical Energy and Slip-Related Costs

The principal energetic quantity used in the high-fidelity experiments is a mechanical terrain-interaction energy. The corresponding raw mechanical power is

$$P_{\text{mech,raw}} = F_{x,L} v_{\text{track},L} + F_{x,R} v_{\text{track},R} + F_{y,L} v_y^b + F_{y,R} v_y^b, \tag{3.20}$$

where the longitudinal terms describe the interaction of each track with the terrain and the lateral terms account for the corresponding body-frame lateral motion. The objective retains positive mechanical power,

$$P_{\text{mech}} = \max\left(0, P_{\text{mech,raw}}\right), \tag{3.21}$$

which is integrated over the complete rollout,

$$E_{\text{mech}} = \int_0^T P_{\text{mech}}(t)\, dt \approx \sum_n P_{\text{mech},n} \Delta t_{\text{sim}}. \tag{3.22}$$

For historical reasons, the implementation stores this quantity under the name `total_energy`. It should nevertheless be interpreted as the mechanical terrain-interaction energy represented by the simulator, rather than

as a complete electrical battery-consumption model.

Two slip-related quantities appear during the evolution of the project and must be distinguished carefully. The historical optimization objective uses

$$P_{\text{slip,signed}} = F_{x,L}\beta_L + F_{x,R}\beta_R + F_{y,L}v_y^b + F_{y,R}v_y^b, \tag{3.23}$$

with accumulated value

$$E_{\text{slip,signed}} = \int_0^T P_{\text{slip,signed}}(t)\, dt. \tag{3.24}$$

Because this quantity is signed, positive and negative contributions can cancel. Consequently, a numerically more negative value cannot automatically be interpreted as a larger reduction in physically dissipated slip energy.

For the later datasets and surrogate models, a non-negative dissipation quantity is preferred. Let $\mathbf{v}_{\text{slip},p}$ denote the relative track–terrain velocity of contact patch p and $\mathbf{F}_p$ its tangential force. The local signed contact power is

$$P_{p,\text{signed}} = \mathbf{F}_p^T \mathbf{v}_{\text{slip},p}, \tag{3.25}$$

from which the non-negative dissipative contribution is defined as

$$P_{p,\text{slip,diss}} = \max\left(0, -P_{p,\text{signed}}\right). \tag{3.26}$$

Summing over the active contact patches and integrating in time yields

$$P_{\text{slip,diss}} = \sum_p P_{p,\text{slip,diss}}, \qquad E_{\text{slip}} = \int_0^T P_{\text{slip,diss}}(t)\, dt. \tag{3.27}$$

The historical signed objective is preserved when analysing experiments that used it, while Equation (3.27) is the preferred physical interpretation for the later teacher datasets.

3.3 Simulator-Backed CHOMP

The high-fidelity optimization branch treats the simulator as a black-box physical cost evaluator inside CHOMP. With fixed start, goal, and duration, the optimization variables are the coordinates of the internal knots,

$$\boldsymbol{\xi}_{\text{int}} = \begin{bmatrix} x_1 & y_1 & \cdots & x_{N-2} & y_{N-2} \end{bmatrix}^T. \tag{3.28}$$

A general objective can be expressed as

$$J(\boldsymbol{\xi}) = J_{\text{physical}}(\boldsymbol{\xi}) + \lambda_{\text{smooth}} J_{\text{smooth}}(\boldsymbol{\xi}). \tag{3.29}$$

The precise physical objective depends on the experiment. Mechanical energy was used as the first high-fidelity objective in order to determine whether a simulator-derived quantity could guide trajectory deformation at all. Smoothness was subsequently included to discourage irregular local changes in the knot sequence. Later surrogate-planning formulations additionally consider slip dissipation, tracking quality, terminal accuracy, and execution duration. A more general planner objective is therefore

$$\begin{aligned} J_{\text{planner}} = {} & \lambda_E E_{\text{mech}} + \lambda_{\text{slip}} E_{\text{slip}} + \lambda_{\text{smooth}} J_{\text{smooth}} \\ & + \lambda_g \Phi_g(e_{\text{goal}}) + \lambda_{\text{track}} \Phi_{\text{track}}(e_{\text{track,max}}) + \lambda_T T. \end{aligned} \tag{3.30}$$

The numerical weights are experiment-specific scientific parameters and are recorded with each configuration rather than transferred silently between experiments. Terminal position error and maximum trajectory-tracking error are defined as

$$e_{\text{goal}} = \left\| \mathbf{p}_{N-1}^{\text{exec}} - \mathbf{p}_{\text{goal}} \right\|, \tag{3.31}$$

and

$$e_{\text{track,max}} = \max_k \left\| \mathbf{p}_k^{\text{exec}} - \mathbf{p}_k^{\text{des}} \right\|. \tag{3.32}$$

These terms are important because reducing a desired-path energy estimate is not useful if the resulting trajectory becomes difficult for the physical vehicle to execute.

3.3.1 Smoothness and Numerical Physical Gradient

The discrete trajectory smoothness cost is represented as

$$J_{\text{smooth}} = \frac{1}{2}\boldsymbol{\xi}_{\text{int}}^T \mathbf{A} \boldsymbol{\xi}_{\text{int}} + \mathbf{b}^T \boldsymbol{\xi}_{\text{int}} + c, \tag{3.33}$$

with analytical gradient

$$\nabla J_{\text{smooth}} = \mathbf{A}\boldsymbol{\xi}_{\text{int}} + \mathbf{b}. \tag{3.34}$$

The high-fidelity physical simulator does not provide an analytical derivative with respect to every trajectory coordinate. A centered finite-difference approximation is therefore used. For internal knot i,

$$\frac{\partial J_{\text{physical}}}{\partial x_i} \approx \frac{J_{\text{physical}}(\boldsymbol{\xi} + \varepsilon \mathbf{e}_{x_i}) - J_{\text{physical}}(\boldsymbol{\xi} - \varepsilon \mathbf{e}_{x_i})}{2\varepsilon}, \tag{3.35}$$

$$\frac{\partial J_{\text{physical}}}{\partial y_i} \approx \frac{J_{\text{physical}}(\boldsymbol{\xi} + \varepsilon \mathbf{e}_{y_i}) - J_{\text{physical}}(\boldsymbol{\xi} - \varepsilon \mathbf{e}_{y_i})}{2\varepsilon}. \tag{3.36}$$

Each internal knot therefore requires four physical perturbation rollouts,

$$x_i + \varepsilon, \quad x_i - \varepsilon, \quad y_i + \varepsilon, \quad y_i - \varepsilon, \tag{3.37}$$

and the total number of finite-difference evaluations is

$$N_{\text{FD}} = 4N_{\text{int}}. \tag{3.38}$$

For the validated configuration with $N_{\text{int}} = 40$,

$$N_{\text{FD}} = 160, \tag{3.39}$$

using

$$\varepsilon = 0.01\,\text{m}. \tag{3.40}$$

If one rollout performs work proportional to $\mathcal{O}(N_{\text{sim}} N_p)$, where N_{sim} is the number of simulation timesteps and N_p the number of active contact patches, then the serial finite-difference gradient has approximate work

$$\mathcal{O}\left(4N_{\text{int}} N_{\text{sim}} N_p\right). \tag{3.41}$$

A complete CHOMP iteration also evaluates the current trajectory and one or more candidate updates. If N_α line-search candidates are tested, the rollout count is approximately

$$N_{\text{rollouts,iter}} \approx 1 + 4N_{\text{int}} + N_\alpha. \tag{3.42}$$

Equation (3.42) explains why distributed physical evaluation is a central part of the method rather than merely a software optimization.

3.3.2 Covariant Update, Line Search, and Recovery

The physical and smoothness contributions are combined into the complete trajectory gradient. The CHOMP search direction is obtained by preconditioning this gradient with the trajectory metric,

$$\mathbf{d} = -\mathbf{A}^{-1} \nabla J. \tag{3.43}$$

A candidate trajectory is then constructed as

$$\boldsymbol{\xi}_{\text{cand}} = \boldsymbol{\xi} + \alpha \mathbf{d}, \tag{3.44}$$

where α is selected through Armijo backtracking. A candidate is accepted when

$$J(\boldsymbol{\xi}_k + \alpha \mathbf{d}_k) \leq J(\boldsymbol{\xi}_k) + c\alpha \nabla J(\boldsymbol{\xi}_k)^T \mathbf{d}_k, \qquad 0 < c < 1, \tag{3.45}$$

otherwise the step is reduced according to

$$\alpha \leftarrow \beta\alpha, \qquad 0 < \beta < 1. \tag{3.46}$$

Because a complete gradient evaluation contains many independent simulator tasks, failures can occur after only part of the wave has completed. Every accepted optimization iteration is therefore checkpointed. If an infrastructure or simulator failure interrupts an iteration, the optimization is restarted from the most recent accepted trajectory using fresh simulator workers. Partial gradients are not reused: the interrupted gradient wave is recomputed completely. This preserves a single coherent optimization history even when one scientific run spans multiple PBS jobs.

The later V4 dataset motivates a possible duration-aware extension in which

$$\boldsymbol{\chi} = \begin{bmatrix} \boldsymbol{\xi}_{\text{int}} \\ T \end{bmatrix} \tag{3.47}$$

becomes the optimization variable. A numerical derivative with respect to duration can be obtained as

$$\frac{\partial J}{\partial T} \approx \frac{J(\boldsymbol{\xi}, T + \varepsilon_T) - J(\boldsymbol{\xi}, T - \varepsilon_T)}{2\varepsilon_T}. \tag{3.48}$$

Such an iteration would require $4N_{\text{int}} + 2$ physical perturbation evaluations. This formulation is reported as a methodological extension motivated by the spatiotemporal dataset; it is not retroactively attributed to the completed fixed-duration high-fidelity experiments.

3.4 Distributed HPC Execution

The computational structure of the finite-difference gradient is particularly well suited to coarse-grained parallelism. Every perturbed trajectory is independent of the others until their resulting costs are assembled into the gradient. The rollout task set can therefore be written as

$$\mathcal{Q} = \{\mathcal{R}_1, \ldots, \mathcal{R}_{4N_{\text{int}}}\}, \tag{3.49}$$

where each $\mathcal{R}_q$ represents one complete physical simulation. The implementation parallelizes these rollout-level tasks rather than attempting to subdivide the numerical integration of one simulator instance.

The runtime follows a controller–worker architecture,

$$\text{controller} \rightarrow \text{task queue} \rightarrow \text{workers} \rightarrow \text{result queue}. \tag{3.50}$$

PBS is responsible for allocating nodes and CPU resources, whereas the application-level controller manages the optimization. The controller owns the current CHOMP trajectory, constructs the finite-difference perturbations, dispatches simulation tasks, reconstructs the gradient, evaluates line-search candidates, and checkpoints accepted states. Each worker owns an isolated simulator context and returns the physical cost together with the diagnostic and provenance information required to reconstruct the experiment.

The architecture does not require the number of allocated workers to equal the number of perturbations. If W healthy workers are available, the finite-difference tasks are completed in approximately

$$N_{\text{waves}} = \left\lceil \frac{4N_{\text{int}}}{W} \right\rceil \tag{3.51}$$

waves. Worker count is therefore treated as an infrastructure parameter: it changes the wall-clock runtime but does not alter the scientific trajectory, finite-difference step, physical objective, or optimizer configuration.

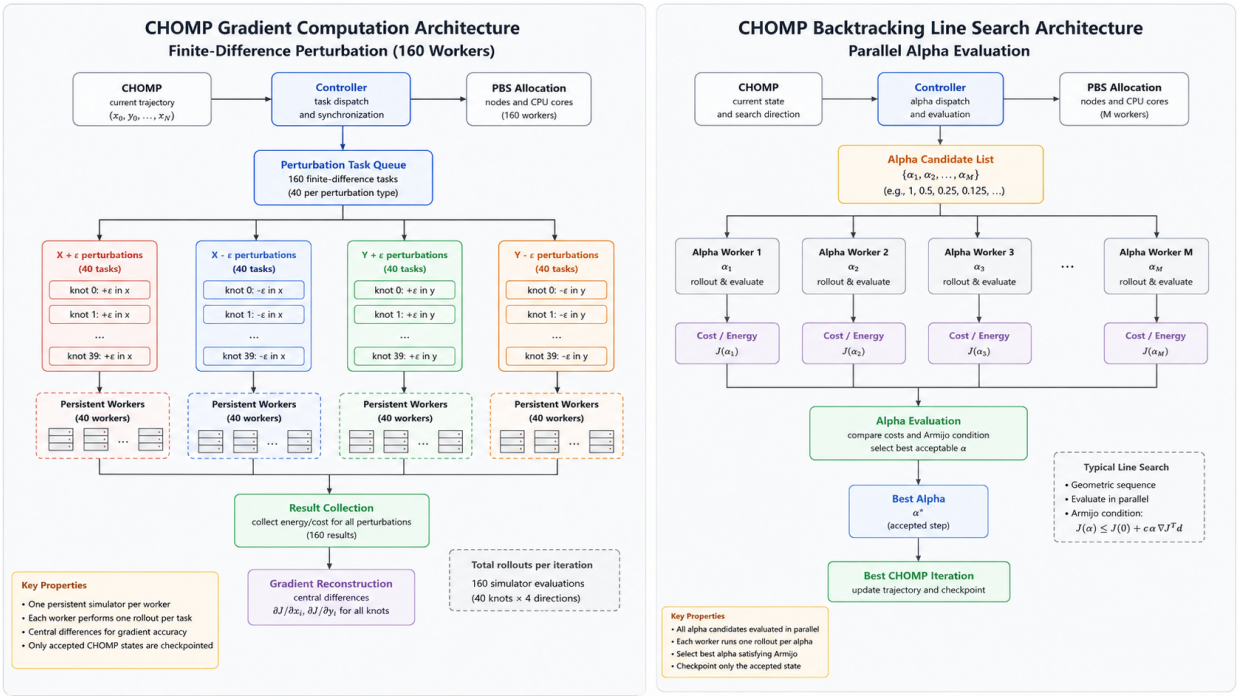


Figure 3.2: Parallel CHOMP architecture for finite-difference gradient reconstruction and candidate-step evaluation. PBS allocates the computing resources, while the application controller distributes independent simulation rollouts, reconstructs the gradient, evaluates candidate updates, and checkpoints accepted iterations.

Repeated initialization of the simulator can itself be expensive, so workers are persistent. Each worker initializes its static environment once and then executes the sequence

$$\text{initialize} \rightarrow \text{rollout} \rightarrow \text{reset} \rightarrow \text{rollout} \rightarrow \cdots \tag{3.52}$$

while retaining only resources that are independent of the current trajectory. Two important reusable components are the serialized robot-model cache and preprocessed terrain data such as cleaned mesh and height-grid representations. Cached and uncached execution are required to remain numerically equivalent; caching changes initialization cost, not the physical model.

Isolation is equally important. Each worker uses independent communication and runtime state, including separate ROS/Gazebo contexts, unique identities, task-specific result files, explicit resets, and bounded cleanup. The purpose is to prevent stale simulator state or failure in one rollout from contaminating another.

The same architecture is reused for dataset generation. In high-fidelity CHOMP, the independent tasks are perturbations of one trajectory. During teacher generation, the natural tasks are instead world–trajectory pairs,

$$(\mathcal{T}_i, \boldsymbol{\xi}_j) \longrightarrow \text{teacher episode}_{ij}. \tag{3.53}$$

This reuse allows the distributed CPU infrastructure to serve both simulator-backed optimization and high-throughput teacher generation, while GPU resources are subsequently used for surrogate-model training.

3.5 Simulator-as-Teacher Dataset Design

The dataset is organized around the Cartesian product

$$\text{World} \times \text{Trajectory}. \tag{3.54}$$

A world contains the terrain geometry, roughness, spatial material fields, generation parameters, identity, and deterministic seed. A trajectory contains its start and goal, desired knots, timing information, generation parameters, and deterministic seed. Combining one world with one trajectory and executing it through the high-fidelity simulator produces a teacher episode containing the executed motion, physical quantities, validity information, and provenance.

An important design principle is the separation between quantities that can be computed from the desired

trajectory or terrain map and quantities that require physical simulation. Terrain height, surface normal, slope, curvature, roughness, material parameters, desired position, heading, velocity, acceleration, yaw rate, path curvature, normalized progress, duration, and knot timing belong to the input/preprocessing side. Teacher outputs include executed position and heading, executed velocities, track commands, mechanical power and energy, slip quantities, contact forces, tracking error, terminal error, and trusted success or failure semantics.

Generalization is evaluated through disjoint terrain worlds,

$$\mathcal{T}_{\text{train}} \cap \mathcal{T}_{\text{val}} = \mathcal{T}_{\text{train}} \cap \mathcal{T}_{\text{test}} = \mathcal{T}_{\text{val}} \cap \mathcal{T}_{\text{test}} = \varnothing. \tag{3.55}$$

This is more demanding than randomly splitting trajectory segments from the same terrain realization because the test set evaluates whether a model can transfer to terrain fields it has never observed during training.

The teacher-generation pipeline also distinguishes physical validity from task success. A rollout can remain physically valid even if the robot does not reach the nominal goal. The dataset therefore separates `teacher_trusted` from `teacher_success`. Trusted soft failures, including tracking-envelope violations, goal-not-reached episodes, and excessive-slip or no-progress cases, can still contain informative physical behaviour. Hard-invalid, incomplete, untrusted, or missing episodes are excluded.

3.5.1 Evolution from V1 to V2

The dataset design evolved as limitations of the earlier distributions became visible. V1 served as a proof of concept. It contained ten terrain worlds and relatively short trajectories, demonstrating that local energetic and motion quantities could be learned from simulator data. However, its mean desired path length of approximately $2.1\,\text{m}$ and maximum of approximately $3.9\,\text{m}$ were substantially smaller than the spatial scale encountered in the later planning experiments. Performance on V1 could therefore not be interpreted as evidence of generalization to planning-scale trajectories.

V2 addressed part of this limitation by formalizing worlds and trajectories as separate reusable assets and by introducing spatially varying material properties. A terrain world was no longer described by geometry alone, but by

$$\mathcal{T} = \{z(x,y), \mu(x,y), c(x,y), K(x,y), \ldots\}\,. \tag{3.56}$$

This design reflects an important property of the physical problem: identical terrain geometry does not imply identical vehicle behaviour when friction, cohesion, or shear-deformation parameters differ. The remaining limitation was coverage. Planning required significantly larger worlds, longer trajectories, and broader variation in both geometry and material properties, motivating the procedural V3 redesign.

3.5.2 V3: Procedural Spatial Planning Distribution

V3 was designed specifically to reduce the distribution mismatch between the early teacher datasets and the spatial scale of trajectory optimization. The canonical planning domain is

$$x, y \in [-20, 20]\ \text{m}, \tag{3.57}$$

corresponding to a $40\,\text{m}\times 40\,\text{m}$ terrain. The stored height grid uses $0.1\,\text{m}$ spacing, producing 401×401 samples when both boundaries are included.

Procedural Terrain Generation

A finite Fourier representation is used to generate the large-scale terrain geometry. The objective is not to assume that real terrain is periodic, but to obtain a controlled and reproducible basis capable of producing smooth hills, valleys, ridges, broad slopes, and combinations of these structures over planning-scale domains.

The macro geometry is

$$z_{\text{macro}}(x,y) = a_x x + a_y y + \sum_{j=1}^{M_z} A_j \cos\left(k_{x,j}x + k_{y,j}y + \phi_j\right), \tag{3.58}$$

where

$$k_{x,j} = k_j \cos\theta_j, \qquad k_{y,j} = k_j \sin\theta_j. \tag{3.59}$$

Each parameter has an interpretable spatial role: A_j controls the height amplitude, k_j determines the wavelength, θ_j determines the orientation of the mode, and ϕ_j shifts it spatially. The linear coefficients a_x and a_y introduce broad global inclination. Combining multiple low-frequency modes therefore produces terrain richer than a single analytical hill while retaining smoothness and bounded spatial frequency.

Roughness is generated independently using a higher-frequency band,

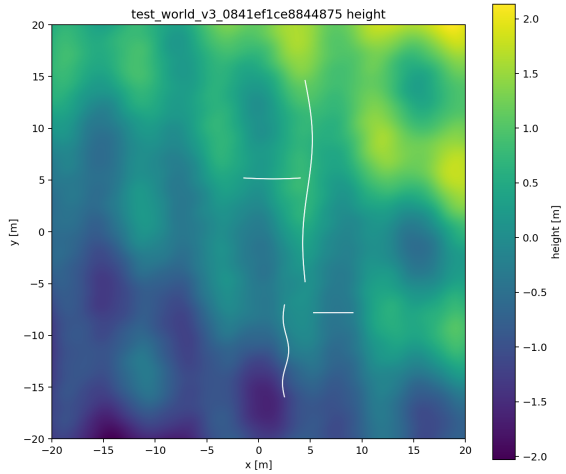
$$z_{\text{rough}}(x, y) = \sum_{j=1}^{M_r} B_j \cos\left(\kappa_{x,j} x + \kappa_{y,j} y + \psi_j\right), \tag{3.60}$$

and the final height field is

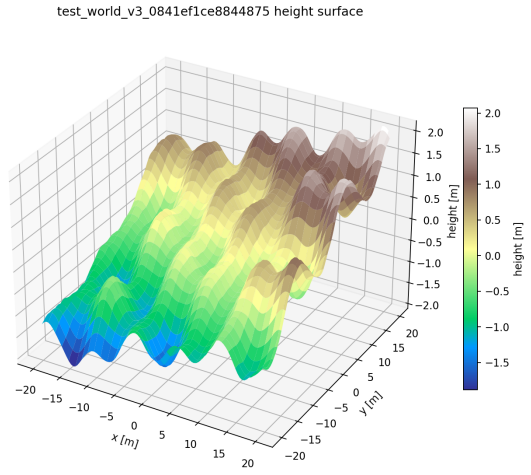
$$z(x, y) = z_{\text{macro}}(x, y) + \alpha_r z_{\text{rough}}(x, y). \tag{3.61}$$

Separating macro geometry from local roughness makes the two effects independently controllable and avoids the discontinuities that would result from adding uncorrelated per-cell white noise.

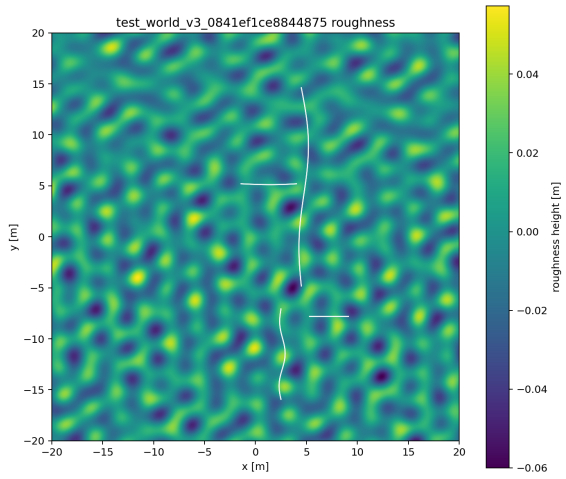
World difficulty is validated from the terrain that was actually generated, rather than inferred solely from the requested Fourier coefficients. For each deterministic seed, the generator samples the active modes, amplitudes, frequencies, directions, phases, and optional global slope; evaluates the resulting fields on the fixed grid; computes measured descriptors such as height range, RMS and maximum slope, and curvature statistics; and rejects worlds that violate simulator-safe limits. The generation parameters and random seed are stored with every accepted world so that it can be recreated exactly.



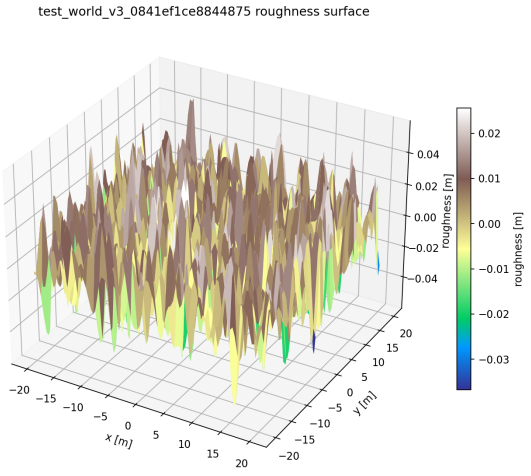
(a) Height map



(b) Height surface



(c) Roughness map



(d) Roughness surface

Figure 3.3: Representative V3 terrain world. The paired two- and three-dimensional views separate the low-frequency macro geometry from the band-limited roughness component.

Spatial Material Fields

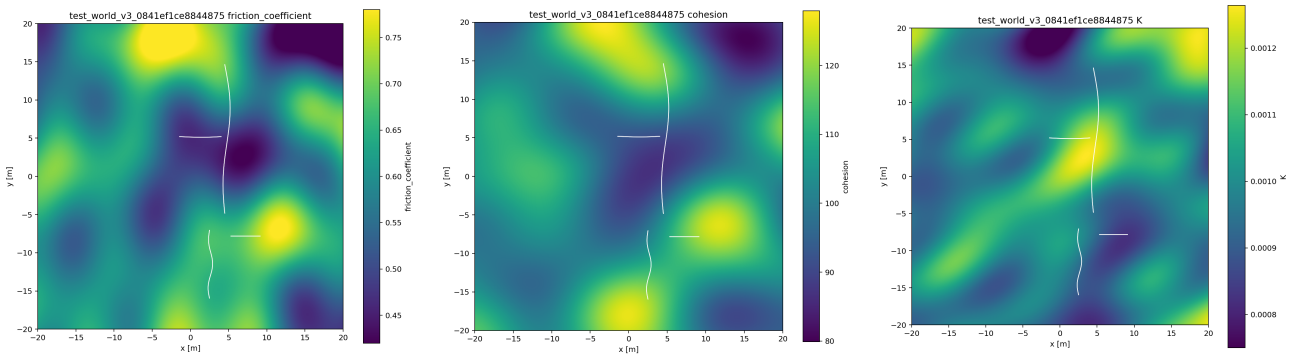
Geometry is only one component of the terrain distribution. V3 also introduces smooth spatial variation in the terramechanical parameters. Each material map is generated from a low-frequency procedural field and normalized into a validated physical interval. For example,

$$\mu(x, y) = \mu_{\min} + \widetilde{F}_{\mu}(x, y)\,(\mu_{\max} - \mu_{\min})\,, \tag{3.62}$$

with validated ranges

$$\mu \in [0.42, 0.78], \tag{3.63}$$
$$c \in [78, 130]\ \text{Pa}, \tag{3.64}$$
$$K \in [0.00075, 0.00135]\ \text{m}. \tag{3.65}$$

A shared low-frequency soil-regime component introduces partial correlation between the parameters, while smaller independent perturbations prevent the maps from becoming identical. The resulting dataset can therefore contain regions that share similar geometry but produce different physical responses.



(a) Friction coefficient (b) Cohesion (c) Shear-deformation modulus K

Figure 3.4: Representative V3 material fields for the same terrain world. Their smooth but non-identical spatial variation ensures that local physical response is not determined by geometry alone.

Trajectory Distribution

Trajectories are generated independently of the terrain assets and then combined with worlds during teacher generation. Start and goal states are sampled broadly across the usable domain, while the internal path is generated in a local start–goal coordinate system. For normalized progress $s \in [0, 1]$, a lateral displacement profile is defined as

$$d(s) = \sum_{j} A_j \sin(j\pi s) + s(1 - s) \sum_{m} b_m (2s - 1)^m. \tag{3.66}$$

Because

$$d(0) = d(1) = 0, \tag{3.67}$$

the generator preserves the selected endpoints while producing a broad range of internal shapes, from nearly straight trajectories to S-shaped and multi-turn paths. Generated candidates are rejected when they violate the world boundary or exceed configured limits on quantities such as curvature, speed, acceleration, yaw rate, yaw acceleration, or self-intersection.

V3 uses

$$T_{\text{V3}} = 50\,\text{s}, \qquad N = 101, \qquad \Delta t_{\text{knot}} = 0.5\,\text{s}. \tag{3.68}$$

The total duration is fixed, but local speed is not necessarily constant: variation is produced by non-uniform spatial spacing of the knots. The final V3 teacher dataset contains 60 terrain worlds and 3000 usable episodes.

3.5.3 V4: Spatiotemporal Planning Distribution

V4 retains the procedural terrain, material-field, and spatial trajectory generation principles of V3 while introducing one additional source of variation: global execution duration. The purpose is to distinguish two trajectories that share the same or similar geometry but are executed at different speeds.

The knot count remains

$$N = 101, \tag{3.69}$$

so that

$$\Delta t_{\text{knot}} = \frac{T}{100}. \tag{3.70}$$

A target mean speed first defines a raw duration,

$$T_{\text{raw}} = \frac{L}{v_{\text{mean,target}}}, \tag{3.71}$$

after which the resulting timing is checked against the configured speed and acceleration limits. The validated continuous speed regimes are centred approximately at

$$0.65\,\text{m/s}, \qquad 0.45\,\text{m/s}, \qquad 0.28\,\text{m/s}, \qquad 0.16\,\text{m/s}. \tag{3.72}$$

V4 therefore contains both forms of timing variation: local variations in speed continue to arise from non-uniform spatial knot spacing, while changing T adds global execution-speed variation.

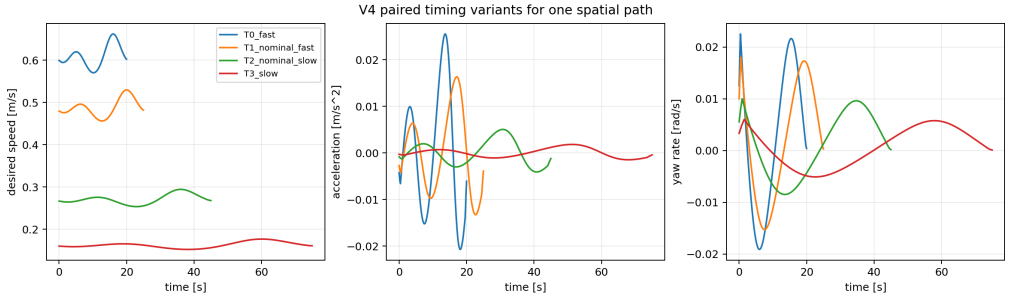


Figure 3.5: Paired V4 timing variants for the same spatial trajectory, showing how speed, acceleration, and yaw-rate profiles change across execution-duration regimes.

The final V4 teacher dataset contains 70 worlds and 7000 usable episodes. The conceptual difference between V3 and V4 is summarized in Table 3.1.

Table 3.1: Conceptual distinction between the V3 and V4 teacher datasets.

Property	V3	V4
Procedural geometry	Yes	Yes
Procedural roughness	Yes	Yes
Spatial material fields	Yes	Yes
Random start/goal pairs	Yes	Yes
Diverse smooth paths	Yes	Yes
Local speed-profile variability	Yes	Yes
Fixed total duration	Yes	No
Variable global duration	No	Yes
Primary question	Spatial generalization	Spatiotemporal generalization

3.6 Surrogate Model Families

The surrogate architectures considered in this work are organized according to the level at which the high-fidelity simulator is approximated. Three main families are investigated: *direct planning models*, which map a complete planning problem directly to either a scalar cost or an executed trajectory; a *local forward model*,

which reconstructs the executed trajectory recursively; and an independent *power model*, which estimates the energetic quantities associated with an executed trajectory.

This separation leads to the following conceptual structure:

$$\begin{aligned}
&\boldsymbol{\xi}_{\text{des}} \longrightarrow \text{Direct Scalar Model} \longrightarrow \hat{J}, \\
&\boldsymbol{\xi}_{\text{des}} \longrightarrow \text{Direct Trajectory Model} \longrightarrow \hat{\boldsymbol{\xi}}_{\text{exec}} \longrightarrow \text{Power Model} \longrightarrow (\hat{E}_{\text{mech}}, \hat{E}_{\text{slip}}), \\
&\boldsymbol{\xi}_{\text{des}} \longrightarrow \text{Local Forward Model} \longrightarrow \hat{\boldsymbol{\xi}}_{\text{exec}} \longrightarrow \text{Power Model} \longrightarrow (\hat{E}_{\text{mech}}, \hat{E}_{\text{slip}}).
\end{aligned} \tag{3.73}$$

Here, $\boldsymbol{\xi}_{\text{des}}$ denotes the desired trajectory and $\hat{\boldsymbol{\xi}}_{\text{exec}}$ an approximation of the trajectory that would be executed by the high-fidelity teacher simulator. The direct scalar approach bypasses explicit reconstruction of the executed motion, whereas the remaining architectures preserve this intermediate physical representation.

3.6.1 Direct Planning Models

The first family attempts to approximate the planning problem directly from the complete desired trajectory and its terrain context. Unlike recursive models, the entire candidate trajectory is processed in one forward pass. Two forms of direct prediction are investigated: scalar prediction and executed-trajectory prediction.

Direct Scalar Prediction

The direct scalar formulation represents the shortest computational path from a candidate trajectory to its planning cost. Rather than reproducing the vehicle motion generated by the terramechanics simulator, the model directly estimates trajectory-level energetic quantities.

Two variants are considered.

Pure learned terrain-conditioned scalar model. The first variant is a fully learned mapping from the desired trajectory, terrain description, and timing information to the corresponding scalar quantities:

$$f_\theta^{\text{scalar}} : (\boldsymbol{\xi}_{\text{des}}, \mathcal{T}, \boldsymbol{\tau}) \longrightarrow \left(\hat{E}_{\text{mech}}, \hat{E}_{\text{slip}}, \hat{J}_{\text{total}} \right), \tag{3.74}$$

where $\mathcal{T}$ denotes the terrain representation associated with the trajectory and $\boldsymbol{\tau}$ contains the relevant temporal information.

The terrain representation may contain geometric and material descriptors such as local elevation, slope, curvature, roughness, and terrain parameters. Consequently, the network learns directly how the interaction between the candidate motion and the underlying terrain affects the final energetic cost.

This architecture is computationally attractive because each candidate trajectory requires only one network evaluation. It is therefore particularly suitable for applications in which a planner must compare many candidate trajectories.

Its main limitation is that the executed vehicle motion is not reconstructed. The network must compress the complete terrain–trajectory interaction into a small number of scalar outputs, and quantities such as tracking error, trajectory deviation, local slip, and terminal state error cannot be recovered from the prediction.

Reduced inverse dynamics with learned terrain correction. The second scalar formulation introduces a physics-derived baseline before applying a learned correction. A reduced inverse-dynamics model first estimates the energetic demand associated with the desired vehicle motion.

For planar skid-steering motion, approximate left- and right-track velocities can be obtained as

$$v_k^L = v_{x,k} - \frac{B}{2}\omega_{z,k}, \qquad v_k^R = v_{x,k} + \frac{B}{2}\omega_{z,k}, \tag{3.75}$$

where B is the track separation, $v_{x,k}$ is the desired longitudinal velocity, and $\omega_{z,k}$ is the desired yaw rate.

The resulting reduced model provides a computationally inexpensive baseline estimate

$$E_{\text{ID}} = f_{\text{ID}}\left(\boldsymbol{\xi}_{\text{des}}, \boldsymbol{\tau}\right). \tag{3.76}$$

Because this approximation does not reproduce the complete track–terrain interaction, a learned model is used to estimate the terrain-dependent difference between the reduced model and the high-fidelity simulator:

$$\Delta\hat{E}_\theta = f_\theta^{\text{corr}}\left(\boldsymbol{\xi}_{\text{des}}, \mathcal{T}, \boldsymbol{\tau}\right). \tag{3.77}$$

The final prediction becomes

$$\hat{E}_{\text{mech}} = E_{\text{ID}} + \Delta\hat{E}_\theta. \tag{3.78}$$

The purpose of this architecture is to avoid asking the neural network to learn the complete energetic relationship from data. Motion-dependent behavior that can already be described by the reduced model is retained explicitly, while the learned component focuses on the terrain-dependent effects that are absent from the simplified physics.

This formulation therefore provides an intermediate approach between a purely analytical approximation and a completely data-driven scalar surrogate.

Direct Executed-Trajectory Prediction

The second direct architecture preserves substantially more information than the scalar formulation. Instead of predicting only the final cost, it predicts the complete trajectory that the high-fidelity simulator would execute.

The desired trajectory is represented by

$$\boldsymbol{\xi}_{\text{des}} = \left[\mathbf{x}_0^{\text{des}}, \mathbf{x}_1^{\text{des}}, \ldots, \mathbf{x}_M^{\text{des}}\right], \tag{3.79}$$

and the direct sequence model approximates

$$f_\theta^{\text{traj}} : \left(\boldsymbol{\xi}_{\text{des}}, \mathcal{T}, \boldsymbol{\tau}\right) \longrightarrow \hat{\boldsymbol{\xi}}_{\text{exec}} = [\hat{\mathbf{x}}_0^{\text{exec}}, \ldots, \hat{\mathbf{x}}_M^{\text{exec}}]. \tag{3.80}$$

The complete sequence is generated in one forward pass. Unlike the local forward model introduced in the following section, a predicted state is therefore not recursively reused as the input to the next prediction.

This avoids recursive accumulation of one-step prediction errors while still providing an explicit approximation of the teacher-executed trajectory. The predicted motion can consequently be evaluated in terms of trajectory RMSE, maximum trajectory deviation, terminal position error, terminal heading error, velocity error, and yaw-rate error.

More importantly, predicting the executed trajectory separates the *motion-prediction problem* from the *energy-prediction problem*. The resulting trajectory can subsequently be evaluated using the same independent power model employed by the local forward architecture:

$$\hat{\boldsymbol{\xi}}_{\text{exec}} \longrightarrow f_\phi^{\text{power}} \longrightarrow \left(\hat{E}_{\text{mech}}, \hat{E}_{\text{slip}}\right). \tag{3.81}$$

This modularity is useful because improvements to either the trajectory model or the power model can be introduced independently.

3.6.2 Local Forward Model

The local forward architecture approximates the dynamics of the high-fidelity teacher at a local temporal scale. Instead of processing the complete trajectory in a single forward pass, the model predicts the state reached at the following label interval.

For each sample, the local transition model receives the current executed state, the desired motion, and descriptors of the terrain surrounding the vehicle:

$$f_\theta^{\text{forward}} : \left(\mathbf{x}_k^{\text{exec}}, \mathbf{u}_k^{\text{des}}, \mathbf{g}_k, \mathbf{m}_k\right) \longrightarrow \hat{\mathbf{x}}_{k+1}^{\text{exec}}, \tag{3.82}$$

where $\mathbf{g}_k$ represents local terrain geometry and $\mathbf{m}_k$ contains the relevant terrain-material descriptors.

During one-step training and validation, the input state $\mathbf{x}_k^{\text{exec}}$ is taken from the teacher trajectory. The model therefore learns the local transition

$$\mathbf{x}_{k+1}^{\text{exec}} \approx f_\theta^{\text{forward}}\left(\mathbf{x}_k^{\text{exec}}, \mathbf{u}_k^{\text{des}}, \mathbf{g}_k, \mathbf{m}_k\right). \tag{3.83}$$

For an actual surrogate rollout, however, the teacher state is no longer available. Predictions must instead be propagated recursively:

$$\hat{\mathbf{x}}_{k+1}^{\text{exec}} = f_\theta^{\text{forward}}\left(\hat{\mathbf{x}}_k^{\text{exec}}, \mathbf{u}_k^{\text{des}}, \mathbf{g}_k, \mathbf{m}_k\right). \tag{3.84}$$

Repeated application of this transition model reconstructs the complete executed trajectory,

$$\hat{\boldsymbol{\xi}}_{\text{exec}} = [\hat{\mathbf{x}}_0^{\text{exec}}, \hat{\mathbf{x}}_1^{\text{exec}}, \ldots, \hat{\mathbf{x}}_M^{\text{exec}}]. \tag{3.85}$$

The local formulation introduces a different inductive bias from the direct trajectory model. Because the same transition function is repeatedly applied, the network is encouraged to learn local vehicle–terrain interactions rather than a fixed mapping between complete trajectory sequences.

This property is particularly relevant to the objective of generalization to previously unseen terrain configurations and trajectory lengths. However, the recursive formulation also introduces error accumulation: even small errors in individual transitions can progressively modify the state presented to later predictions.

For this reason, one-step performance alone is not sufficient to characterize the model. Evaluation must also consider recursive trajectory RMSE, maximum deviation, terminal position error, terminal heading error, and the evolution of prediction error over the complete rollout.

Once the executed trajectory has been reconstructed, energetic quantities are not predicted directly by the forward model. Instead, the trajectory is passed to the same power model used for the direct trajectory architecture:

$$\hat{\boldsymbol{\xi}}_{\text{exec}} \longrightarrow f_\phi^{\text{power}} \longrightarrow \left(\hat{E}_{\text{mech}}, \hat{E}_{\text{slip}}\right). \tag{3.86}$$

3.6.3 Unified Power Model

The power model constitutes an independent component of the surrogate architecture. Its purpose is not to predict vehicle motion, but to estimate the physical quantities associated with an already specified or predicted executed trajectory.

This separation is important because the source of the executed trajectory can vary. During training and validation, the power estimator can be evaluated on the trajectory generated by the high-fidelity teacher. During surrogate inference, the same estimator can instead operate on the trajectory predicted by either the direct trajectory model or the local forward model.

The three cases can therefore be summarized as

$$\begin{aligned} \boldsymbol{\xi}_{\text{exec}}^{\text{teacher}} &\longrightarrow f_\phi^{\text{power}}, \\ \hat{\boldsymbol{\xi}}_{\text{exec}}^{\text{direct}} &\longrightarrow f_\phi^{\text{power}}, \\ \hat{\boldsymbol{\xi}}_{\text{exec}}^{\text{forward}} &\longrightarrow f_\phi^{\text{power}}. \end{aligned} \tag{3.87}$$

At every label interval, the power model uses the local executed motion and terrain context to estimate the relevant physical quantities:

$$f_\phi^{\text{power}} : (\mathbf{x}_k^{\text{exec}}, \mathbf{u}_k, \mathbf{g}_k, \mathbf{m}_k) \longrightarrow \left(\hat{P}_{\text{mech},k}, \hat{P}_{\text{slip},k}\right). \tag{3.88}$$

Depending on the available simulator labels, the output vector may additionally contain quantities such as local forces, velocities, slip-related quantities, or other physically meaningful intermediate variables. Mechanical and slip energies are obtained by temporal integration:

$$\hat{E}_{\text{mech}} = \sum_{k=0}^{M-1} \hat{P}_{\text{mech},k} \Delta t_{\text{label},k}, \qquad \hat{E}_{\text{slip}} = \sum_{k=0}^{M-1} \hat{P}_{\text{slip},k} \Delta t_{\text{label},k}. \tag{3.89}$$

A planning objective can subsequently be constructed from these quantities, for example as

$$\hat{J} = w_{\text{mech}} \hat{E}_{\text{mech}} + w_{\text{slip}} \hat{E}_{\text{slip}} + J_{\text{other}}, \tag{3.90}$$

where the weights depend on the optimization objective and J_{other} represents any additional geometric or

smoothness terms that remain analytically evaluated by the planner.

The principal advantage of this decomposition is that trajectory prediction and energetic prediction can be validated independently. For example, the power model can first be evaluated using the true teacher-executed trajectory, isolating errors in physical-quantity prediction. The complete surrogate pipeline can then be evaluated using predicted trajectories, where the final energy error contains contributions from both motion prediction and power prediction.

The architecture therefore provides a common energetic representation across the different surrogate families while preserving a clear distinction between

$$\underbrace{\text{Where does the vehicle move?}}_{\text{trajectory model}} \quad \text{and} \quad \underbrace{\text{How much energy does that motion require?}}_{\text{power model}}. \tag{3.91}$$

3.7 Evaluation Protocol and Metrics

The learned models developed in this work are intended not only to reproduce individual simulator outputs, but ultimately to support trajectory evaluation and optimization. Consequently, evaluation is performed at several complementary levels: prediction accuracy, accumulated physical quantities, trajectory fidelity, execution feasibility, candidate ranking, and computational efficiency.

Throughout this thesis, the validated high-fidelity terramechanics simulator is treated as the *teacher* and therefore provides the reference values against which reduced and learned models are evaluated. Whenever possible, the most important results are reported separately for validation terrain and for previously unseen test worlds, since generalization across terrain is more relevant to planning than interpolation among trajectories generated on the same terrain.

Not every metric applies to every model. Scalar models are primarily evaluated through regression and ranking metrics; trajectory and local-forward models additionally require geometric and state-prediction metrics; power models require both instantaneous and integrated-energy evaluation; and surrogate planners must ultimately be assessed through the physical quality of the trajectory obtained after optimization.

3.7.1 Prediction Accuracy for Scalar and Time-Series Quantities

Consider a reference quantity y_i produced by the high-fidelity simulator and the corresponding model prediction $\hat{y}_i$. This notation applies both to scalar outputs, such as trajectory energy, and to samples of time-varying physical quantities, such as mechanical power, slip power, velocity, or force.

The mean absolute error is defined as

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \,. \tag{3.92}$$

Because it retains the physical units of the predicted quantity, MAE provides the most direct measure of the typical magnitude of the prediction error.

The root mean squared error is

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}. \tag{3.93}$$

RMSE gives greater weight to large errors and is therefore useful for detecting occasional but significant prediction failures that may be hidden by the average absolute error.

To compare errors across quantities or trajectories with different characteristic magnitudes, a stabilized relative error is used:

$$e_{\text{rel},i} = \frac{|y_i - \hat{y}_i|}{\max(|y_i|, \epsilon_{\text{rel}})}, \tag{3.94}$$

where $\epsilon_{\text{rel}} > 0$ prevents numerical amplification when the reference value is close to zero. Relative errors are therefore interpreted together with absolute errors rather than in isolation.

Systematic over- or under-prediction is quantified through the bias

$$\text{Bias} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i). \tag{3.95}$$

This quantity is particularly relevant for locally predicted quantities that are later integrated along a trajectory, since even a small persistent bias can accumulate into a significant final energy error.

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{3.96}$$

where $\bar{y}$ denotes the mean reference value. An R^2 close to one indicates that the model explains most of the observed variation in the target, while $R^2 = 0$ corresponds to the performance of a constant mean predictor. Negative values indicate prediction performance worse than this baseline.

Finally, the median absolute error is reported as a robust measure of typical behaviour, while the maximum absolute error characterizes the worst observed prediction. The latter is particularly important for quantities related to trajectory feasibility or severe slip, for which rare large errors can be more consequential than average performance.

3.7.2 Accumulated Energy and Slip Metrics

Power models require an additional level of evaluation because small local errors can accumulate over time. If the predicted power at sample k is $\hat{P}_k$, the corresponding trajectory-level energy estimate is obtained by numerical integration:

$$\hat{E} = \sum_{k=1}^{M} \hat{P}_k \Delta t_k. \tag{3.97}$$

The same procedure is applied to the simulator power sequence,

$$E = \sum_{k=1}^{M} P_k \Delta t_k, \tag{3.98}$$

allowing the trajectory-level energy error to be written as

$$e_E = \hat{E} - E. \tag{3.99}$$

Across the evaluation dataset, integrated energy is therefore assessed using absolute error, RMSE, relative error, and bias. When the corresponding quantities are available, this procedure is performed separately for total mechanical energy and slip-related energy:

$$E_{\text{mech}}, \qquad E_{\text{slip}}. \tag{3.100}$$

This distinction is important because a model may accurately predict total energy while incorrectly representing the contribution caused specifically by track–terrain slip. Conversely, good pointwise power predictions do not automatically guarantee accurate trajectory-level energy estimates if a small systematic error accumulates over long horizons.

3.7.3 Trajectory and State-Prediction Metrics

Models predicting vehicle motion are evaluated against the trajectory actually executed by the high-fidelity teacher rather than only against the desired geometric path.

Let

$$\mathbf{p}_k = \begin{bmatrix} x_k & y_k \end{bmatrix}^{\mathsf{T}} \tag{3.101}$$

denote the teacher position and $\hat{\mathbf{p}}_k$ its predicted counterpart. The trajectory root mean squared error is

$$\mathrm{RMSE}_{\mathrm{traj}} = \sqrt{\frac{1}{M}\sum_{k=1}^{M} |\hat{\mathbf{p}}_k - \mathbf{p}_k|_2^2}. \tag{3.102}$$

This metric measures the overall geometric divergence accumulated throughout the rollout.

The terminal position error is

$$e_{\mathrm{term,pos}} = |\hat{\mathbf{p}}_M - \mathbf{p}_M|_2 \,, \tag{3.103}$$

while terminal heading error is evaluated using the wrapped angular difference

$$e_{\mathrm{term},\psi} = \left|\mathrm{wrap}_{[-\pi,\pi)}\left(\hat{\psi}_M - \psi_M\right)\right| . \tag{3.104}$$

The largest instantaneous geometric discrepancy is captured by

$$e_{\max} = \max_k |\hat{\mathbf{p}}_k - \mathbf{p}_k|_2 \,. \tag{3.105}$$

This complements trajectory RMSE because two rollouts may have similar average error while one contains a much larger local deviation.

For models predicting the complete vehicle state, the same principle is extended to longitudinal and lateral velocity, yaw rate, heading, and other available state variables. Their MAE and RMSE are reported individually so that trajectory error can be related to the physical state component responsible for the divergence.

For autoregressive local-forward models, these quantities are evaluated both over individual prediction steps and over recursively generated multi-step rollouts. The distinction exposes error accumulation: good one-step accuracy does not necessarily imply stable long-horizon prediction.

3.7.4 Execution Feasibility

Low energy is useful only when the corresponding trajectory remains physically executable. Feasibility prediction is therefore treated as an additional component of model evaluation.

A particularly undesirable failure is a *false-safe* prediction, in which the surrogate considers a trajectory feasible while the high-fidelity teacher exhibits loss of tracking, numerical or physical failure, or another predefined infeasibility condition. Conversely, a *false reject* occurs when an executable trajectory is incorrectly classified as infeasible.

The false-safe rate is defined as

$$r_{\mathrm{FS}} = \frac{N_{\text{pred. feasible, teacher failed}}}{N_{\text{teacher failed}}}, \tag{3.106}$$

while the false-reject rate is

$$r_{\mathrm{FR}} = \frac{N_{\text{pred. failed, teacher feasible}}}{N_{\text{teacher feasible}}}. \tag{3.107}$$

Where explicit success/failure labels are available, accuracy, precision, recall, and the corresponding confusion matrix are additionally reported. However, classification metrics alone are insufficient. Terminal position error, maximum tracking error, and trajectory divergence are also examined because feasibility can degrade continuously before a hard failure criterion is reached.

The asymmetry between the two error types is important in trajectory optimization. A false reject removes a potentially useful candidate from consideration; a false safe can instead cause an optimizer to favour a trajectory that cannot be reproduced by the physical reference model. For this reason, the false-safe rate receives particular attention.

3.7.5 Planning and Candidate-Ranking Metrics

The final purpose of several surrogate models is trajectory optimization. Consequently, accurate prediction of absolute cost is not sufficient: the surrogate must also preserve the relative ordering among nearby candidate trajectories.

This property is evaluated through Spearman rank correlation, pairwise ordering accuracy, and top-k agree-

ment between the surrogate and high-fidelity teacher. Pairwise ordering accuracy measures the fraction of trajectory pairs for which both models agree on which candidate has the lower physical cost.

These metrics are particularly relevant to CHOMP, where the optimizer repeatedly compares local trajectory modifications. A surrogate may therefore remain useful even when its predictions contain a moderate absolute offset, provided that the local cost ordering and gradient direction are sufficiently preserved.

Nevertheless, ranking metrics remain intermediate indicators. The definitive planning evaluation is performed by taking the final trajectory produced by surrogate-based optimization and executing it again using the high-fidelity simulator.

For each surrogate-planning experiment, the comparison therefore includes

- final high-fidelity mechanical energy;
- final high-fidelity slip energy or slip-related cost;
- trajectory geometry and tracking behaviour;
- improvement relative to the common initialization;
- optimizer convergence history;
- total planning wall time;
- number of high-fidelity simulator evaluations required.

When comparing high-fidelity and surrogate-backed CHOMP, the initial trajectory, objective definition, and relevant optimization parameters are kept consistent wherever possible. Since CHOMP is a local optimizer, the optimized trajectories are interpreted relative to their initialization rather than as estimates of a global optimum.

The central validation principle is therefore

> A surrogate-planned trajectory is considered physically improved only after the final candidate has been re-evaluated with the high-fidelity simulator.

3.7.6 Computational Efficiency

The motivation for introducing parallel execution and learned surrogates is to reduce the computational cost associated with high-fidelity physical evaluation.

For a single trajectory evaluation, let t_{teacher} denote the high-fidelity simulator runtime and t_{model} the surrogate inference time. Their ratio defines the evaluation speedup

$$S_{\text{eval}} = \frac{t_{\text{teacher}}}{t_{\text{model}}}. \tag{3.108}$$

However, single-call inference speed does not directly represent planning performance. End-to-end optimization comparisons therefore additionally report

$$t_{\text{plan}}, \qquad N_{\text{sim}}, \qquad N_{\text{model}}, \tag{3.109}$$

where t_{plan} is total optimizer wall time, N_{sim} is the number of high-fidelity simulator calls, and N_{model} is the number of surrogate evaluations.

This distinction separates the computational advantage of an individual model evaluation from the practical speedup obtained by the complete planning algorithm.

3.7.7 Visual Evaluation

Quantitative metrics are complemented by direct visualization because scalar errors alone cannot reveal all relevant physical or geometric failure modes.

For representative validation and unseen-world test episodes, trajectory-prediction models are visualized using the three distinct paths

$$\text{desired/reference trajectory} \quad \text{vs} \quad \text{teacher executed trajectory} \quad \text{vs} \quad \text{model-predicted trajectory}. \tag{3.110}$$

This separation is essential because the desired trajectory is an input to the physical system, whereas the teacher trajectory represents its actual simulated response.

For time-varying physical quantities, teacher and model predictions are plotted over a common time axis. Depending on the model, these quantities include

$$\begin{aligned} &P_{\text{mech}}, \quad P_{\text{slip}}, \quad E_{\text{mech}}, \quad E_{\text{slip}}, \\ &\mathbf{v}, \quad \boldsymbol{\omega}, \quad \mathbf{a}, \quad \mathbf{F}, \quad e_{\text{track}}, \quad J_{\text{local}}. \end{aligned} \tag{3.111}$$

Prediction-error curves are included where they help identify temporal drift or localized failure.

Scalar outputs additionally use predicted-versus-teacher scatter plots and error distributions. These plots reveal systematic bias, heteroscedasticity, outliers, and terrain-dependent failure modes that may not be evident from aggregate metrics.

For trajectory-optimization experiments, visualization extends beyond the final scalar cost. The initial, representative intermediate, and final CHOMP trajectories are compared directly on the corresponding terrain. Convergence plots include total cost, physical energy, slip contribution, smoothness contribution, accepted and rejected iterations, accepted line-search step size, gradient norm, and trajectory-update magnitude where available. This permits evaluation of both optimizer convergence and the geometric evolution of the solution.

3.8 Methodological Synthesis

The methodology developed in this thesis can be interpreted as a set of complementary routes sharing the same physical reference model. The high-fidelity simulator provides the teacher behaviour, while distributed computation and learned surrogate models provide different mechanisms for reducing the cost of repeated physical evaluation.

3.8.1 High-Fidelity Optimization Route

The reference optimization pipeline operates directly on the terramechanics simulator:

$$\begin{aligned} \text{terrain} + \text{desired trajectory} &\rightarrow \text{high-fidelity simulation} \\ &\rightarrow \text{physical trajectory and cost} \\ &\rightarrow \text{finite-difference gradient} \\ &\rightarrow \text{CHOMP} \rightarrow \text{optimized trajectory}. \end{aligned} \tag{3.112}$$

The distributed implementation does not modify the physical model or the optimization objective. It changes only how independent simulator evaluations are executed:

$$\begin{aligned} \text{trajectory perturbations} &\rightarrow \text{controller–worker infrastructure} \\ &\rightarrow \text{parallel simulator rollouts} \\ &\rightarrow \text{cost collection} \\ &\rightarrow \text{gradient reconstruction}. \end{aligned} \tag{3.113}$$

After the gradient has been reconstructed, candidate step sizes can similarly be evaluated through parallel rollout of the line-search trajectories before the next CHOMP iterate is selected.

3.8.2 Teacher Dataset Generation

The same simulator is reused offline to construct the datasets required by the learned models:

$$\begin{aligned} \text{procedural terrain worlds} + \text{desired trajectories} &\rightarrow \text{high-fidelity teacher} \\ &\rightarrow \text{executed states and physical quantities} \\ &\rightarrow \text{V3/V4 datasets} \rightarrow \text{surrogate training}. \end{aligned} \tag{3.114}$$

The generated data retain the distinction between the desired trajectory, the simulator-executed trajectory, and physical quantities derived from the track–terrain interaction. Terrain geometry and other quantities computable directly from the environment are treated as model inputs or preprocessing features rather than simulator-generated labels.

3.8.3 Surrogate Modelling Routes

The learned approaches are organized into three main modelling routes.

The first route consists of *direct planners*. A direct scalar model maps trajectory and terrain information directly to a trajectory-level physical cost:

$$(\mathcal{T}_{\text{des}}, \mathcal{G}) \rightarrow \hat{J}, \tag{3.115}$$

where $\mathcal{T}_{\text{des}}$ represents the candidate desired trajectory and $\mathcal{G}$ the associated terrain description. This family includes both purely terrain-based cost estimation and reduced-physics formulations in which simplified inverse-dynamics information is corrected using terrain-dependent features.

The direct-trajectory alternative predicts the physical response of the vehicle at the trajectory level:

$$(\mathcal{T}_{\text{des}}, \mathcal{G}) \rightarrow \hat{\mathcal{T}}_{\text{exec}}. \tag{3.116}$$

Physical costs can then be estimated from the predicted executed motion rather than directly from the desired path.

The second route is the *local forward model*. Instead of predicting an entire trajectory in a single operation, it approximates the local state transition

$$(\mathbf{x}_k, \mathbf{u}_k, \mathbf{g}_k) \rightarrow \hat{\mathbf{x}}_{k+1}, \tag{3.117}$$

or a short multi-step extension of this mapping. Recursive application reconstructs the predicted executed trajectory and provides an explicit approximation of the vehicle dynamics.

The third route is the *power model*. This component is deliberately kept conceptually independent from the trajectory predictor. Given the relevant state, motion, control, and terrain information, it estimates local physical power quantities:

$$(\mathbf{x}_k, \mathbf{u}_k, \mathbf{g}_k) \rightarrow \left(\hat{P}_{\text{mech},k}, \hat{P}_{\text{slip},k}\right). \tag{3.118}$$

The power model can therefore be evaluated using teacher-executed states, combined with a direct-trajectory model, or composed with the recursive local-forward model. Its trajectory-level outputs are obtained through integration:

$$\hat{\mathcal{T}}_{\text{exec}} \rightarrow \hat{P}(t) \rightarrow \hat{E}, \tag{3.119}$$

providing a common physical-cost representation across different surrogate architectures.

3.8.4 Surrogate-Assisted Planning and Physical Validation

Once trained, the surrogate models replace selected high-fidelity evaluations inside the planning loop:

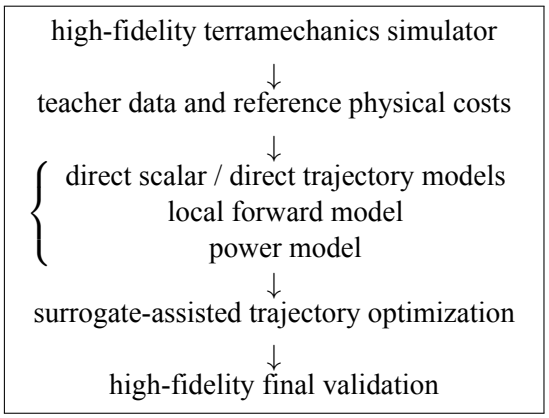
$$\begin{aligned} \text{candidate trajectories} &\rightarrow \text{surrogate evaluation} \\ &\rightarrow \text{surrogate cost or predicted execution} \\ &\rightarrow \text{trajectory optimization} \\ &\rightarrow \mathcal{T}^*_{\text{surrogate}}. \end{aligned} \tag{3.120}$$

The final candidate is then returned to the high-fidelity simulator:

$$\mathcal{T}^*_{\text{surrogate}} \rightarrow \text{high-fidelity validation} \rightarrow \left(E_{\text{mech}}, E_{\text{slip}}, \mathcal{T}_{\text{exec}}, \text{feasibility}\right). \tag{3.121}$$

This final evaluation closes the methodological loop. The surrogate is not treated as a replacement for the validated terramechanics model when establishing physical results; rather, it acts as an approximation that can reduce the number and cost of high-fidelity evaluations required during search.

The complete methodology can therefore be summarized by a single hierarchy:



(3.122)

In this organization, parallel computing and machine learning serve different but complementary purposes. Distributed execution accelerates the use of the original physical simulator without changing its scientific meaning, whereas surrogate modelling approximates selected mappings within that simulator to reduce the number or cost of physical evaluations. In both cases, the high-fidelity terramechanics model remains the common physical reference against which the resulting trajectories, costs, and computational gains are ultimately assessed.

4 Experimental Results

This chapter evaluates the two complementary planning strategies developed in this thesis. The first retains the high-fidelity terramechanics simulator directly inside CHOMP and therefore represents the physical reference solution. The second replaces part of this expensive evaluation with reduced or learned surrogate models trained from simulator-generated data.

The experiments are deliberately evaluated at more than one level. For the exact optimizer, a reduction in scalar cost is not considered sufficient: the initial, intermediate, and final trajectories must also be compared geometrically. For the learned models, conventional held-out regression accuracy is complemented by executed-trajectory prediction and, finally, by planning-oriented diagnostics.

This distinction is central to the interpretation of the chapter:

$$\boxed{\text{prediction accuracy} \neq \text{planning usefulness}}. \tag{4.1}$$

A surrogate can reproduce the average value of a physical quantity accurately while still failing to preserve the local differences between nearby trajectories that determine the direction of a gradient-based optimizer.

4.1 Experimental Questions and Evaluation Strategy

The experimental campaign addresses four related questions.

1. **Physical optimization.** Can simulator-derived energetic objectives produce meaningful terrain-dependent trajectory deformation when used inside CHOMP?

2. **Local optimization behaviour.** How strongly do initialization, travel direction, terrain geometry, and local convergence affect the solution obtained by the exact optimizer?

3. **Surrogate prediction.** Can reduced and learned models reproduce the energetic quantities and executed motion produced by the high-fidelity teacher, including on unseen terrain worlds?

4. **Planning usefulness.** Do improvements in supervised prediction accuracy translate into a useful local cost landscape for surrogate-backed CHOMP?

The simulator is the common physical reference throughout all four questions. In the exact experiments it is called directly by CHOMP. In the learning experiments it generates the teacher labels against which the surrogate models are trained and evaluated.

For this reason, the chapter is organized from the most physically direct experiment to the most approximate one. High-fidelity optimization is presented first, followed by teacher-dataset validation, supervised surrogate prediction, and finally exact-versus-surrogate planning diagnostics.

4.2 High-Fidelity Simulator-Backed CHOMP

The purpose of the exact experiments is to determine whether a trajectory can be deformed according to a cost obtained from the full tracked-vehicle simulator. These runs also provide the reference against which later surrogate-based optimization must be judged.

The canonical configuration is summarized in Table 4.1.

Table 4.1: Canonical high-fidelity CHOMP configuration.

Quantity	Value
Total trajectory knots	42
Internal optimization knots	40
Finite-difference perturbation	$\epsilon = 0.01\,\mathrm{m}$
Physical perturbation rollouts per gradient	160
Smoothness weight	$\lambda_{\mathrm{smooth}} = 200$
Maximum iterations	25
Start and goal	Experiment dependent
Physical objective	Mechanical energy or historical signed slip objective

The evaluation of each run combines scalar convergence with trajectory geometry. In addition to the initial and final objective values, the analysis considers accepted and rejected iterations, displacement from the initialization, path length, turning behaviour, and where available the evolution of the trajectory over intermediate iterations. This is important because CHOMP is a local optimizer: two trajectories can achieve similar cost reductions through very different geometric deformations.

4.2.1 Canonical Total-Energy Experiments

Three canonical experiments, denoted E1–E3, were performed with different initial conditions and trajectory geometries. Their main scalar results are reported in Table 4.2.

Table 4.2: Canonical exact-CHOMP mechanical-energy results.

Experiment	Initialization	Accepted	Rejected	Initial J / Final J
E1	Straight, hill-start case	12	1	$1276.160/347.051$
E2	S-shaped initialization	2	1	$2180.419/399.948$
E3	Straight initialization	25	0	$3430.305/690.456$

All three runs substantially reduce the physical objective, but their convergence histories differ considerably. E2 accepts only two updates, whereas E3 completes all twenty-five iterations. This difference is consistent with the local nature of CHOMP: the gradient available at one initialization does not generally lead to the same local minimum as the gradient obtained from another trajectory.

E1 provides a compact example of this behaviour. Starting from a straight trajectory through a region of strong height variation, the optimizer accepts twelve deformations and reduces the objective from approximately 1276.160 to 347.051 before the final candidate is rejected.

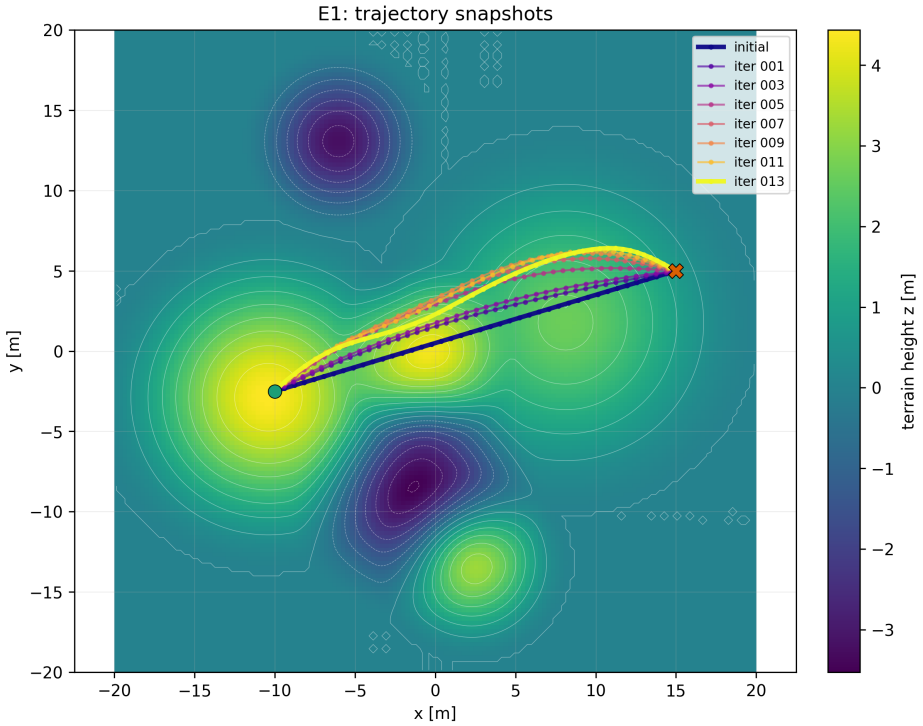


Figure 4.1: E1 initial, intermediate, and final trajectory states over the terrain. The straight initialization is progressively deformed as the optimizer searches for a lower-energy route between the fixed endpoints.

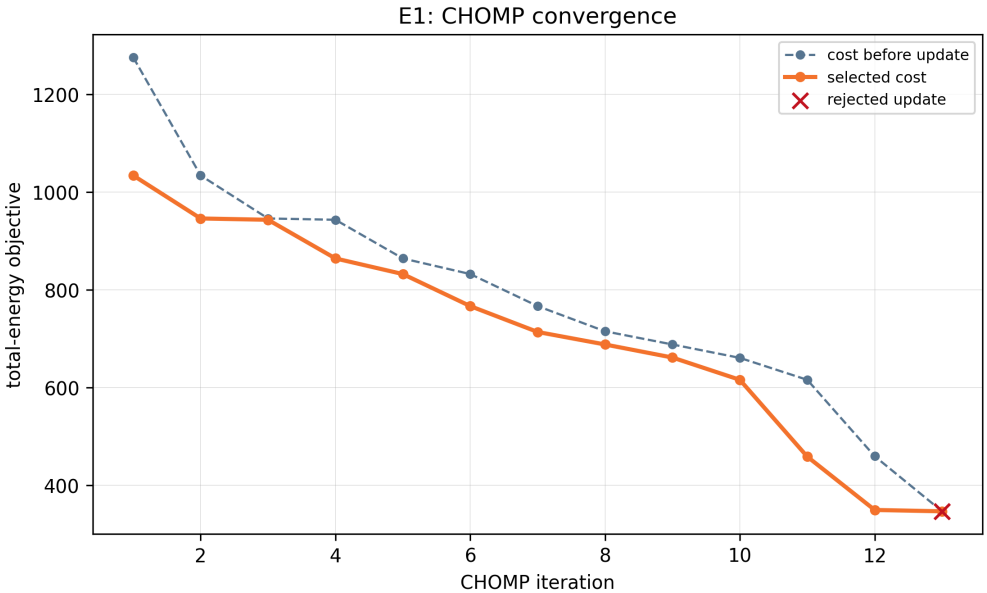


Figure 4.2: E1 physical-cost convergence. Accepted updates progressively reduce the objective, while the final rejected candidate indicates local termination under the tested search direction and line-search parameters.

The trajectory and convergence plots must be interpreted together. The scalar reduction establishes that the physical objective decreases, while the trajectory snapshots verify that this reduction corresponds to a coherent geometric modification rather than an isolated numerical change.

E3 provides the longest complete reference run. It uses terrain `terrain_chen2`, start position $[-19, -5]$ m, goal position $[15, 5]$ m, a straight initialization, and mechanical energy as the physical objective.

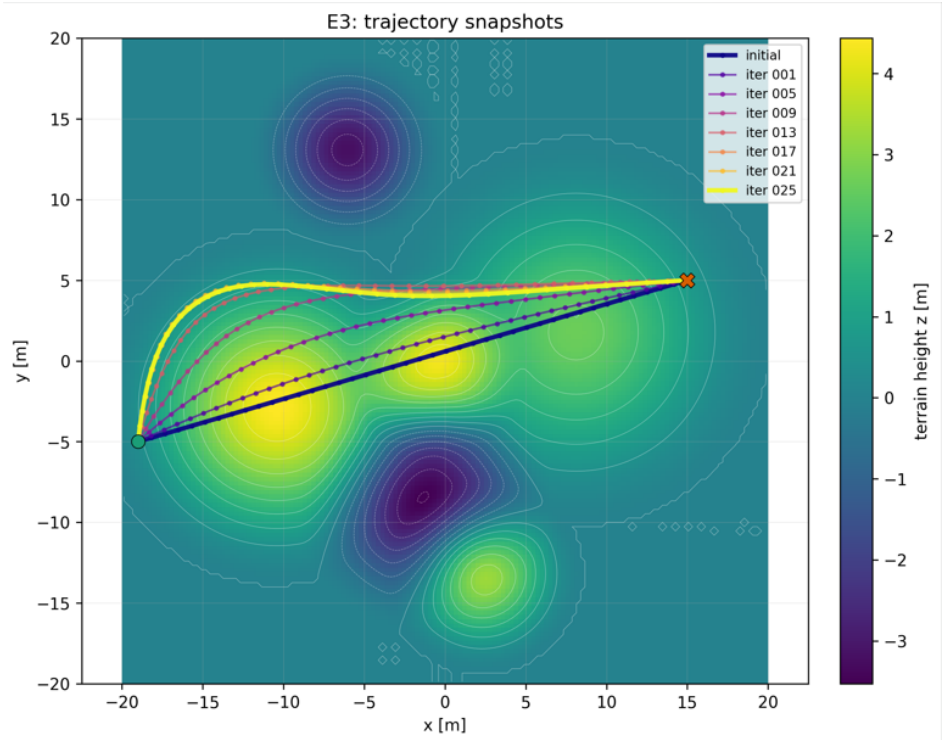


Figure 4.3: E3 initial, representative intermediate, and final trajectories over the terrain.

The objective decreases from approximately 3430.3 to 690.5. More importantly, the final trajectory is not merely a small perturbation of the initial path. Relative to the initialization, the exact high-fidelity deformation reaches approximately

$$d_{\text{mean}} \approx 5.31\,\text{m}, \qquad d_{\text{max}} \approx 8.95\,\text{m}. \tag{4.2}$$

The metre-scale displacement observed in E3 later provides an important reference for evaluating whether surrogate CHOMP produces a physically meaningful search direction.

4.2.2 Terrain Geometry and Direction Dependence

The high-to-low, low-to-high, and hill-avoidance experiments were selected to expose physical effects that cannot be understood from scalar convergence alone.

The high-to-low trajectory connects $[0, 0]$ to $[10, -2]$ m. Its straight initialization has path length

$$L_{\text{init}} \approx 10.198\,\text{m}, \tag{4.3}$$

whereas after six accepted updates the optimized trajectory has

$$L_{\text{final}} \approx 10.794\,\text{m}. \tag{4.4}$$

The objective decreases from 223.430 to 2.484, while the recorded mechanical-energy task decreases from approximately 223.430 to 1.572. The geometric displacement is

$$d_{\text{mean}} \approx 0.844\,\text{m}, \qquad d_{\text{max}} \approx 1.282\,\text{m}, \tag{4.5}$$

and the final path has mean absolute heading change of approximately 0.050 rad and maximum absolute heading change of approximately 0.124 rad.

This result directly demonstrates the difference between geometric and physical optimality. The optimized trajectory is roughly $0.60\,\mathrm{m}$ longer than its initialization, yet the high-fidelity physical energy becomes much smaller because the robot follows a different region of the terrain.

Reversing the geometric endpoints produces a very different result. The low-to-high case connects $[10, -2]$ to $[0, 0]$ m and begins with an objective of 1237.331. None of the tested candidates provides a trusted decrease, so the trajectory remains unchanged. This run is classified as optimizer non-acceptance or local convergence rather than simulator or HPC failure.

The contrast between the two directions is physically meaningful. Reversing a path does not reverse gravity, slope loading, track–terrain interaction, or the energetic consequences of climbing and descending. The physical planning problem is therefore not symmetric under endpoint exchange.

The hill-avoidance case provides a second example of large geometric deformation. It connects $[-15, -12]$ to $[-10, 5]$ m and accepts twenty-four updates before a final rejection. The objective decreases from 2284.184 to 381.257.

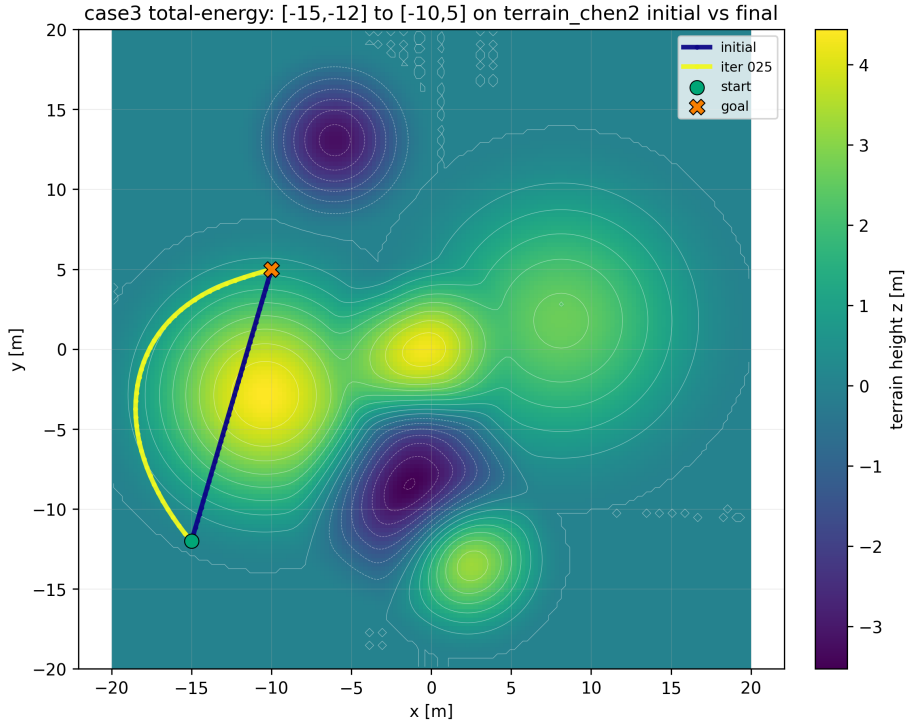


Figure 4.4: Hill-avoidance initial, intermediate, and final trajectories over the terrain.

Relative to its initialization, the high-fidelity final trajectory reaches

$$d_{\mathrm{mean}} \approx 3.88\,\mathrm{m}, \qquad d_{\mathrm{max}} \approx 6.05\,\mathrm{m}. \tag{4.6}$$

Together, the E3 and hill-avoidance cases establish the scale of deformation that the physical optimizer can produce when a strong terrain-dependent gradient is present.

4.2.3 Historical Signed Slip Objective

A separate E3 experiment uses the historical `slip_energy` objective. The optimization value changes from

$$-631.142 \quad \text{to} \quad -1549.432 \tag{4.7}$$

after three accepted iterations followed by one rejection.

This result is intentionally not interpreted as a percentage reduction in physical slip dissipation. The historical objective is signed, so positive and negative contributions can cancel and a more negative value does not necessarily correspond to greater non-negative dissipated energy.

A physically meaningful comparison between energy-optimized and slip-optimized trajectories therefore requires both final trajectories to be re-evaluated using the same non-negative teacher quantities introduced in Chapter 3.

4.2.4 Checkpoint/Resume and Computational Performance

The exact experiments also validate the recovery and distributed-execution methodology. During the original E3 lineage, execution was interrupted while the gradient of iteration 10 was being computed, after nine accepted iterations. The incomplete gradient was not reused. Instead, the optimizer resumed from the last accepted trajectory with fresh workers and recomputed the complete iteration-10 gradient. The original and resumed PBS jobs therefore form one logical scientific run rather than two separate optimizations.

The computational cost remains substantial even with persistent parallel workers. A representative high-to-low run used 99 workers across 11 compute hosts. Across seven gradient evaluations, the run executed 1120 finite-difference tasks and 1155 simulator calls in total. Mean current-cost evaluation time was approximately 74.4 s, mean gradient wall time 179.4 s, and mean line-search time 87.7 s.

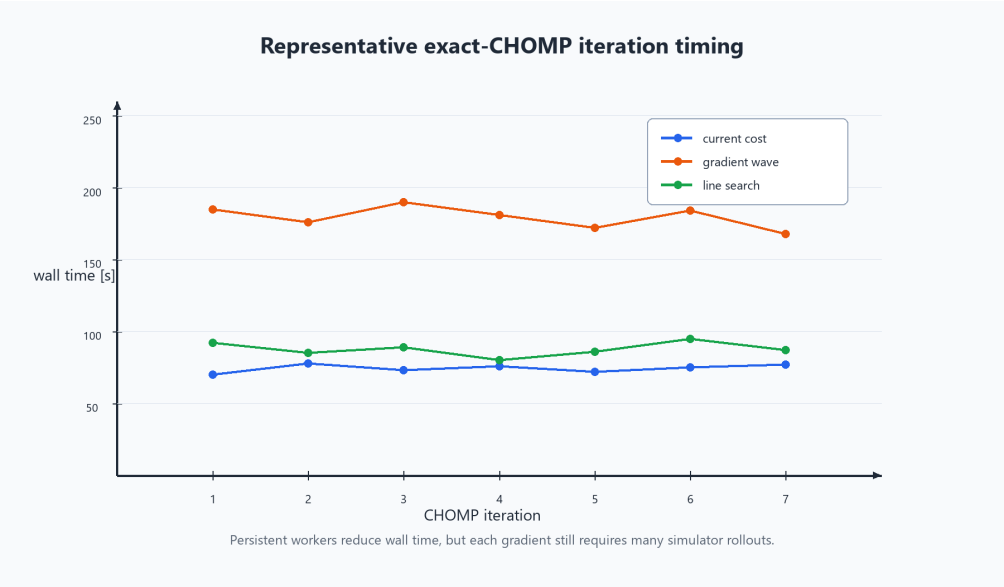


Figure 4.5: Representative wall-clock decomposition of exact simulator-backed CHOMP, including current-cost evaluation, distributed finite-difference gradient waves, and line-search evaluation.

These results demonstrate the benefit and the limitation of the HPC approach. Parallelism substantially reduces the latency of the finite-difference wave, but it does not reduce the number of physical evaluations required by the optimizer. This remaining simulator-call cost motivates the surrogate branch of the thesis.

4.3 Teacher Dataset Coverage and Quality

The surrogate experiments depend critically on the distribution from which the teacher examples are generated. Dataset construction is therefore not treated only as preprocessing; its coverage must be evaluated experimentally.

The first V1 dataset established that local physical quantities could be learned, but its trajectories were much shorter than those encountered during planning. The mean desired path length was approximately 2.1 m and the maximum approximately 3.9 m. By contrast, the V3 pilot reached an average path length of approximately 18.0 m and a maximum of approximately 36.4 m.

This difference changes the learning problem qualitatively. A model that performs well on short local trajectories can still fail when a planner asks for predictions along long, curved paths crossing several terrain regimes. The V3/V4 redesign therefore attempts to bring the supervised distribution closer to the spatial and temporal scales queried during optimization.

4.3.1 Dataset Accounting and Generalization Splits

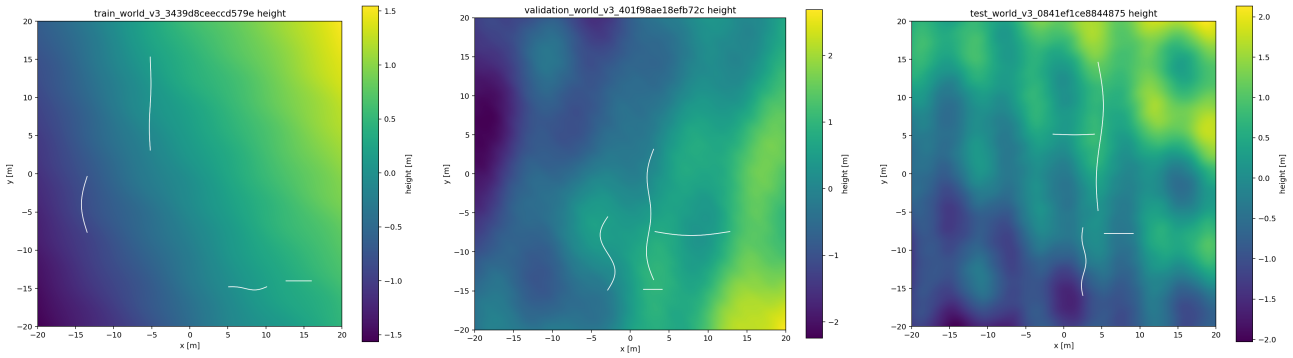
The final V3 dataset contains 60 procedural worlds and 3000 usable teacher episodes, consisting of 2851 trusted successes and 149 trusted soft failures. The final V4 dataset contains 70 worlds and 7000 usable episodes, with 6614 trusted successes and 386 trusted soft failures.

Table 4.3: Final V3/V4 teacher-dataset accounting.

Dataset	Usable episodes	Trusted success	Trusted soft failure
V3	3000	2851	149
V4	7000	6614	386

Retaining trusted soft failures is important for planning. A physically valid rollout in which the robot fails to reach the requested goal is evidence of a region of poor tracking or physical infeasibility. Removing such episodes would bias the dataset toward trajectories that were already easy to execute.

Training, validation, and test sets are separated by terrain world. The representative examples in Figure 4.6 illustrate the corresponding variation in height structure and path placement.



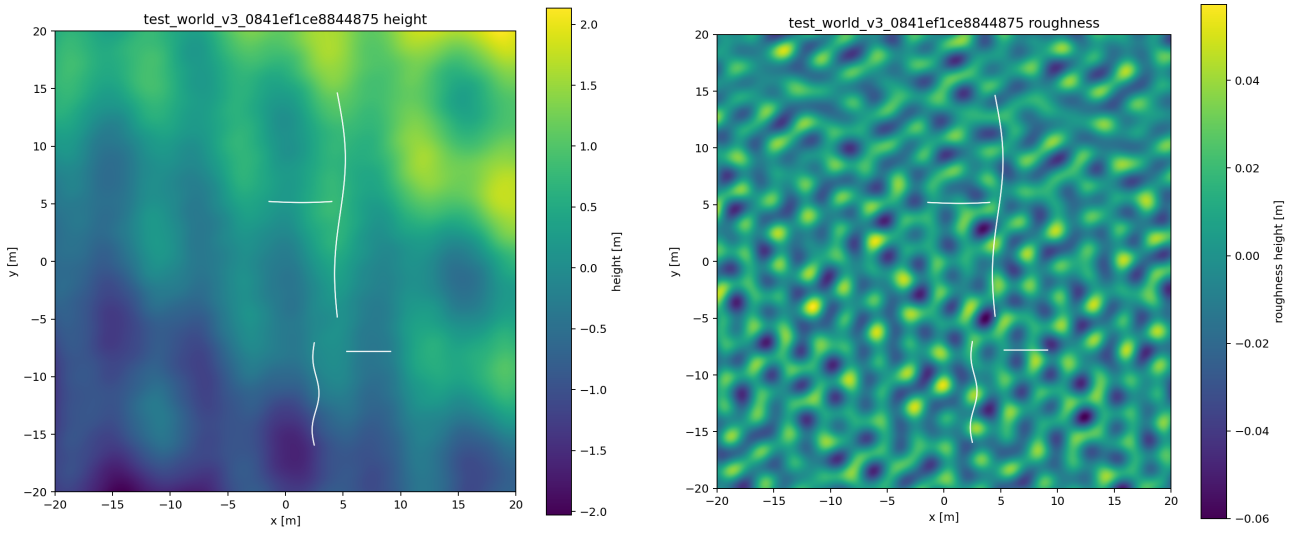
(a) Training world (b) Validation world (c) Test world

Figure 4.6: Representative height fields from the world-separated V3 splits. Although generated from the same procedural family, the terrain realizations and path placements remain distinct.

The split therefore tests generalization to terrain fields not encountered during training rather than merely to unseen segments from an already known world.

4.3.2 Spatial and Material Diversity

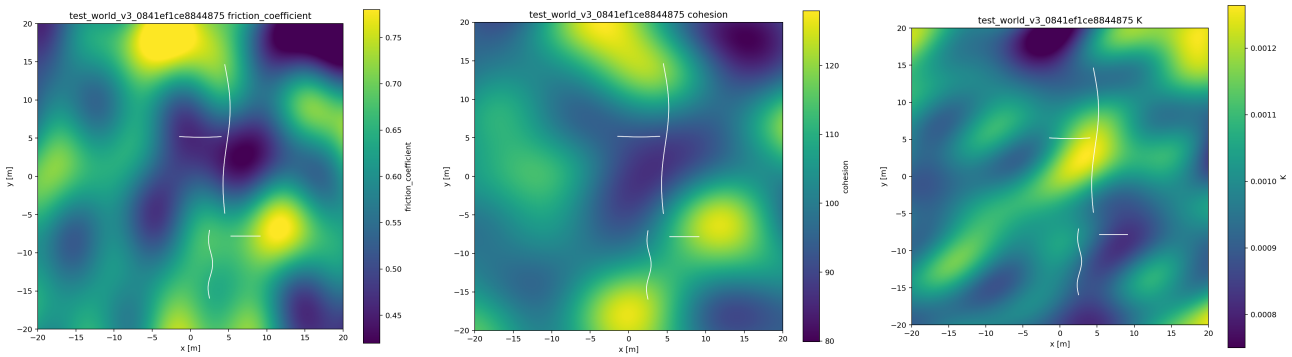
A representative V3 terrain is shown in Figure 4.7. The large-scale height component controls broad hills, valleys, and slopes, whereas the band-limited roughness component adds smaller spatial variations.



(a) Macro height field (b) Local roughness field

Figure 4.7: Representative V3 test-world fields with generated paths overlaid. Macro geometry and local roughness form complementary components of the same terrain realization.

Geometry alone does not define the physical world. Friction, cohesion, and shear-deformation modulus vary spatially and modify the forces produced by the same nominal terrain shape.



(a) Friction coefficient (b) Cohesion (c) Shear-deformation modulus K

Figure 4.8: Representative material fields from one V3 terrain realization. Their smooth but distinct spatial variation exposes the teacher and learned models to terrain response that cannot be inferred from height alone.

This diversity is especially important for CHOMP-oriented learning because the optimizer queries locally perturbed trajectories. The training distribution must therefore cover not only many complete paths, but also a sufficiently broad region around the kinds of paths that the optimizer is likely to deform.

4.3.3 Temporal Diversity in V4

V4 adds global execution duration as an additional dimension of the teacher distribution. Similar spatial paths can therefore be executed under different timing regimes, changing the associated velocity, acceleration, yaw-rate, tracking, slip, and energetic profiles.

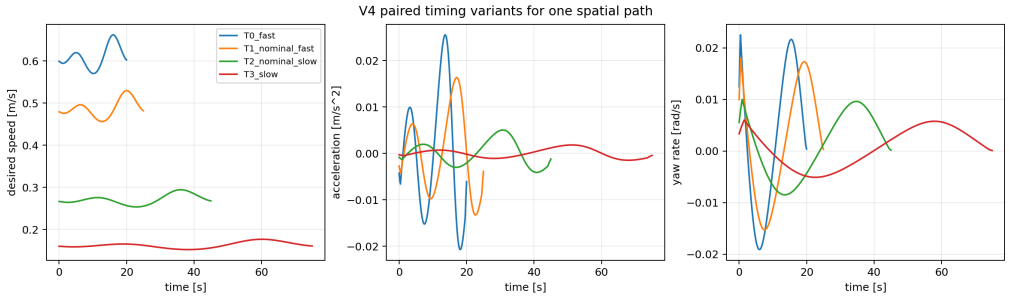


Figure 4.9: Representative V4 timing variants for one spatial trajectory. Speed, acceleration, and yaw-rate profiles change as the same geometry is executed under different global duration regimes.

The learning question is consequently extended from purely spatial generalization to spatiotemporal generalization: the model must account not only for where the robot travels, but also for how rapidly the trajectory is executed.

4.4 Surrogate Prediction Results

The surrogate experiments are presented in two stages. The V1 results compare different approximation families under the original compact dataset and therefore reveal which physical quantities are easier or harder to predict. The final V3/V4 campaign then evaluates matched model families on larger, world-separated planning-scale datasets.

4.4.1 V1: Reduced Physics and Learned Models

The reduced inverse-dynamics model provides the cheapest physical baseline. Its overall prediction accuracy is limited,

Table 4.4: V1 Reduced-ID prediction performance.

Metric	Mechanical power	Mechanical energy
MAE	12.292 W	72.928 J
RMSE	22.559 W	103.761 J
R^2	0.044	-0.039

but the aggregate metrics hide strong geometry dependence. Straight trajectories reach energy $R^2 = 0.989$ with MAE $5.03\,\mathrm{J}$; gentle turns decrease to $R^2 = 0.220$ with MAE $42.95\,\mathrm{J}$; and S-curves reach $R^2 = -5.158$ with MAE $174.47\,\mathrm{J}$. The reduced model therefore captures simple longitudinal motion well but misses important terrain and track-interaction effects during stronger turning.

Its computational advantage is nevertheless substantial. The complete set of 200 episodes requires approximately $1.82\,\mathrm{s}$ of reduced-model computation versus approximately $1953\,\mathrm{s}$ of recorded teacher simulation time, corresponding to a speedup on the order of 10^3.

Learned local-power models recover much more of the teacher variation.

Table 4.5: V1 direct local-power results across multiple training seeds.

Model	Power MAE [W]	Power RMSE [W]	Power R^2	Energy MAE [J]
Transformer	3.483 ± 0.517	6.158 ± 0.802	0.934 ± 0.017	10.988 ± 2.220
GRU	4.463 ± 0.314	7.276 ± 0.474	0.908 ± 0.012	13.327 ± 1.342
ResMLP	4.743 ± 0.583	8.657 ± 0.814	0.870 ± 0.025	16.313 ± 1.824

The Transformer obtains the strongest local-power result and an integrated energy $R^2 = 0.983 \pm 0.006$. A physics-plus-residual ResMLP also improves substantially over the reduced baseline,

$$\mathrm{MAE}_P = 5.331 \pm 1.249\ \mathrm{W}, \qquad R^2_P = 0.848 \pm 0.047, \tag{4.8}$$

with integrated energy MAE

$$\mathrm{MAE}_E = 22.521 \pm 9.058\ \mathrm{J}. \tag{4.9}$$

The direct learned model is therefore more accurate in V1, whereas the residual formulation retains an explicit reduced-physics component.

The trajectory-prediction task exposes a different challenge. Local-forward models may achieve extremely small one-step errors while accumulating larger errors during recursive rollout.

Table 4.6: V1 autoregressive local-forward results.

Model	Rollout RMSE [m]	Terminal position [m]	Maximum deviation [m]
LSTM	0.0462 ± 0.0108	0.0781 ± 0.0192	0.0838 ± 0.0218
TCN	0.0639 ± 0.0277	0.1059 ± 0.0536	0.1140 ± 0.0523
GRU	0.1133 ± 0.0370	0.2006 ± 0.0613	0.2082 ± 0.0581

For the best LSTM, one-step position RMSE is approximately 0.0020 ± 0.0004 m and one-step yaw RMSE 0.0018 ± 0.0006 rad, whereas terminal heading error over the complete autoregressive rollout reaches approximately 0.0749 ± 0.0473 rad. This difference demonstrates why a forward model intended for planning must be evaluated recursively rather than only through one-step regression.

The comparison of matched V1 pipelines is summarized in Table 4.7.

Table 4.7: Matched V1 surrogate pipelines.

Pipeline	Power MAE [W]	Power R^2	Energy MAE [J]	Traj. RMSE [m]
Reduced-ID	9.768	0.312	55.92	–
Reduced-ID + ResMLP	5.331	0.848	22.52	–
Direct Transformer	3.484	0.934	10.99	–
Oracle state-power	3.736	0.919	12.44	–
LSTM + state-power	3.609	0.924	11.90	0.0756

The direct Transformer is the strongest scalar energetic predictor in this matched comparison. The forward-plus-power pipeline is slightly less direct, but additionally predicts the executed trajectory and can therefore support tracking and terminal-state objectives that cannot be recovered from one scalar energy value alone.

4.4.2 Matched V3/V4 Comparison

The final V3/V4 campaign uses matched prediction tasks and comparable model families so that the effect of the enlarged data distribution can be examined without unnecessarily changing the underlying learning problem.

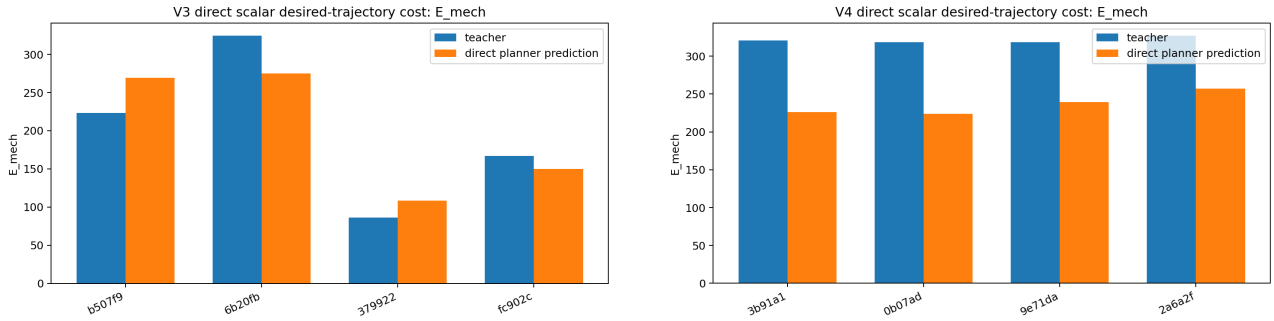
Table 4.8: Best task-specific V3/V4 model results from the matched campaign. RMSE values are meaningful only within the same prediction task because different rows use different targets and physical units.

Task	V3 best model / RMSE	V4 best model / RMSE	Comparison
Local power	Transformer-small / 2.661	ResMLP / 3.065	V3 lower
Direct scalar	ResMLP / 50.214	Transformer-small / 39.670	V4 lower
Direct trajectory	GRU / 0.261	GRU / 0.242	V4 lower
Local forward	GRU / 6.17×10^{-4}	GRU / 4.50×10^{-4}	V4 lower

The results are not a uniform V4 improvement. V3 remains slightly better for local-power prediction, while V4 improves direct scalar, direct trajectory, and local-forward RMSE. Because V4 covers a wider timing distribution, comparable or improved error despite the more diverse task provides evidence that the models have learned part of the dependence on execution duration rather than relying exclusively on the fixed V3 timing regime.

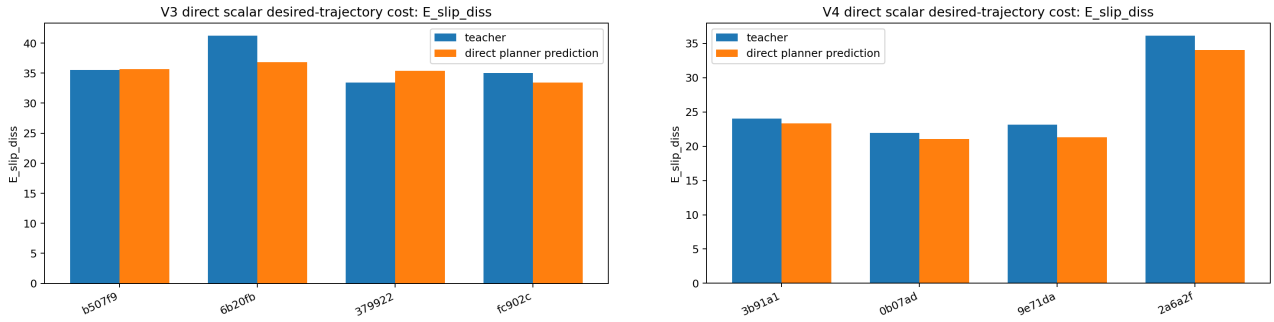
4.4.3 Representative Physical and Trajectory Predictions

Aggregate metrics alone do not show whether the predictions preserve the structure required by a planner. Representative examples are therefore used to inspect both physical quantities and executed trajectories.



(a) V3 (b) V4

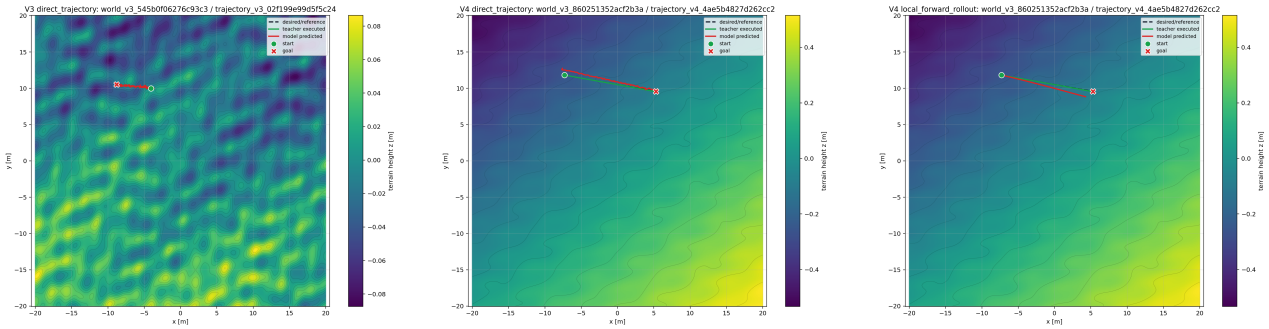
Figure 4.10: Representative direct-scalar mechanical-energy predictions compared with high-fidelity teacher values for V3 and V4.



(a) V3 (b) V4

Figure 4.11: Representative direct-scalar slip-dissipation predictions compared with high-fidelity teacher values for V3 and V4.

Direct-scalar models are attractive because a complete trajectory can be compressed into a small set of planning-relevant physical values with one model evaluation. However, the scalar representation does not expose how the robot moves while producing that cost. Trajectory-prediction models are therefore evaluated separately.



(a) V3 direct trajectory (b) V4 direct trajectory (c) V4 local-forward rollout

Figure 4.12: Representative trajectory predictions. Desired/reference, teacher-executed, and model-predicted trajectories are shown separately; the local-forward case also illustrates the accumulation of transition error during recursive rollout.

These figures preserve the three trajectory concepts used throughout the thesis:

$$\boxed{\text{desired trajectory} \neq \text{teacher-executed trajectory} \neq \text{model-predicted trajectory.}} \tag{4.10}$$

A planning surrogate must preserve not only scalar energetic quantities but also, when required by the objective, the geometric consequences of imperfect execution.

4.5 From Prediction Accuracy to Planning Usefulness

The strongest result of the surrogate campaign is that improved held-out prediction does not automatically produce a useful local CHOMP objective.

In an earlier V3 direct-scalar planning diagnostic, both E3 and hill-avoidance trajectories completed twenty-five accepted surrogate updates. Despite this apparently successful optimizer history, the actual geometric motion was extremely small: mean knot displacement was only approximately $0.015\,\mathrm{m}$.

This scale is fundamentally different from the corresponding exact high-fidelity deformations.

Table 4.9: Exact high-fidelity versus V3 direct-scalar surrogate trajectory deformation.

Case	Method	Mean displacement	Maximum displacement
E3	Exact high-fidelity	$5.31\,\mathrm{m}$	$8.95\,\mathrm{m}$
E3	V3 direct scalar	$\approx 0.015\,\mathrm{m}$	–
Hill avoidance	Exact high-fidelity	$3.88\,\mathrm{m}$	$6.05\,\mathrm{m}$
Hill avoidance	V3 direct scalar	$\approx 0.015\,\mathrm{m}$	–

The later V4 models improve several supervised prediction metrics, especially for direct scalar, direct trajectory, and local-forward prediction. However, the tested surrogate-CHOMP trajectories still deform only at millimetre-to-centimetre scale rather than the metre scale observed under the physical optimizer.

The experimental implication is therefore

$$\boxed{\text{low held-out error} \not\Rightarrow \text{correct local cost landscape} \not\Rightarrow \text{correct optimization trajectory.}} \tag{4.11}$$

This does not contradict the supervised results. Standard MAE, RMSE, or R^2 quantify prediction error over a distribution of examples. CHOMP, instead, compares a set of closely related trajectories around the current iterate. Small systematic prediction errors that contribute little to global RMSE can flatten, distort, or even reverse these local differences.

Planning therefore requires an additional validation question:

> Does the surrogate preserve the relative physical cost of the small trajectory perturbations that determine the optimizer's search direction?

This is stricter than predicting the absolute physical cost of independently sampled trajectories.

The exact and surrogate branches should consequently be interpreted as complementary rather than interchangeable. Exact simulator-backed CHOMP provides the expensive physical reference. Surrogate CHOMP offers the possibility of much cheaper evaluation, but any final trajectory produced by the surrogate must be replayed through the high-fidelity simulator before its physical energy, slip, tracking quality, or feasibility can be accepted as a planning result.

4.6 Discussion and Limitations

Several conclusions emerge when the exact and surrogate experiments are considered together.

First, the high-fidelity experiments show that geometric length is not a sufficient proxy for energetic quality. The high-to-low case becomes longer while dramatically reducing the simulator-derived mechanical-energy objective. Terrain selection and direction therefore matter physically.

Second, the exact optimization results confirm the strongly local nature of CHOMP. E1, E2, E3, high-to-low, low-to-high, and hill avoidance exhibit different accepted-step histories and different scales of deformation. Different initializations, endpoint directions, and terrain regions expose different local objective landscapes; none of these experiments establishes global optimality.

Third, distributed execution changes the feasibility of exact experimentation but not its computational complexity. Persistent workers and parallel finite-difference waves reduce wall-clock latency, yet hundreds of high-fidelity rollouts are still required by one optimization iteration.

Fourth, the surrogate experiments demonstrate that the appropriate model architecture depends on the quantity being predicted. Reduced inverse dynamics is extremely fast but loses important curved-motion interaction. Direct sequence models are effective energetic regressors, while recurrent models are better suited to recursive state prediction. V4 improves several spatiotemporal prediction tasks but does not automatically solve the planning-gradient problem.

These conclusions must be interpreted within several limitations. The terramechanics simulator is the highest-fidelity model used in the thesis but is not real-world ground truth. The historical signed slip objective should not be interpreted as non-negative physical dissipation. V1 has substantially narrower spatial coverage than the final planning problem. Finally, surrogate-planning claims remain valid only after the candidate trajectory has been evaluated again with the high-fidelity teacher; supervised test metrics alone are not sufficient evidence of physical improvement.

4.7 Chapter Summary

The experiments establish both the potential and the limitations of physics-aware trajectory optimization for the tracked vehicle considered in this thesis.

High-fidelity simulator-backed CHOMP is capable of producing large, terrain-dependent trajectory deformations and can identify solutions whose mechanical-energy cost differs substantially from that of a geometrically shorter path. The experiments also show that the resulting optimization is strongly local and computationally expensive, even when the finite-difference rollouts are executed in parallel.

The surrogate results address this cost from a different direction. Learned models substantially improve over simple reduced physics for predicting teacher power, energy, and executed motion, while the V3/V4 datasets extend evaluation toward larger spatial domains, unseen terrain worlds, spatially varying material properties, and variable execution timing.

The final planning diagnostics nevertheless expose a more demanding requirement. A model intended to replace the simulator inside CHOMP must reproduce not only the teacher's average outputs, but also the local variation of physical cost under small trajectory perturbations. The tested models achieve promising predictive accuracy but do not yet reproduce the metre-scale trajectory deformation observed with the exact physical objective. For this reason, high-fidelity simulation remains the final reference for assessing surrogate-optimized trajectories.

5 Conclusions

This thesis investigated whether physically meaningful energy costs can be incorporated into trajectory optimization for tracked robots without making the planning process impractically expensive. Two complementary directions were studied. First, the high-fidelity terramechanics simulator was used directly inside CHOMP as a physically grounded evaluator. Second, the same simulator was treated as a teacher for reduced and learned surrogate models intended to approximate executed motion, mechanical power, and trajectory-level energy at a lower computational cost.

The experiments demonstrate that energy-aware trajectory optimization is both meaningful and technically feasible. They also reveal that replacing the simulator inside an optimizer is considerably more demanding than obtaining accurate supervised predictions. Simulator-backed CHOMP produced large and physically interpretable trajectory deformations, whereas the evaluated surrogates did not reproduce the local cost structure required to guide CHOMP reliably. The principal conclusion is therefore that a planning surrogate must preserve the differences and ordering between neighbouring trajectory perturbations, rather than only achieving a low average error on an independent test set.

5.1 Summary of Findings and Contributions

The work used a pre-existing high-fidelity tracked-vehicle simulator developed at the University of Trento as its physical reference. The simulator represents distributed track–terrain interaction through multiple contact patches and combines terrain geometry, spatial material properties, accumulated shear displacement, shear stress, contact forces, and rigid-body dynamics. The contribution of this thesis was not the development of that simulator, but its integration into a complete trajectory-optimization and simulator-as-teacher learning pipeline.

In the exact optimization branch, the physical objective was differentiated numerically because analytical derivatives of the complete simulator were unavailable. With 40 internal trajectory knots and perturbations of both planar coordinates, one centered finite-difference gradient requires

$$4 \times 40 = 160 \tag{5.1}$$

complete simulator rollouts, before the additional evaluations required by the Armijo line search. This computational workload motivated a distributed controller–worker architecture on the UniTrento HPC cluster. Persistent and isolated workers evaluated gradient perturbations and line-search candidates, while checkpoint and resume support preserved accepted optimizer states and ensured that interrupted gradient waves could be recomputed consistently.

The exact CHOMP experiments confirmed the physical motivation of the thesis. The objectives in experiments E1, E2, and E3 decreased respectively from 1276.160 to 347.051, from 2180.419 to 399.948, and from 3430.305 to 690.456. These reductions were accompanied by substantial geometric changes. In E3, the mean and maximum knot displacements relative to the initialization were approximately $5.31\,\mathrm{m}$ and $8.95\,\mathrm{m}$; in the hill-avoidance case, they were approximately $3.88\,\mathrm{m}$ and $6.05\,\mathrm{m}$. The optimized high-to-low path was approximately $0.60\,\mathrm{m}$ longer than its straight initialization while requiring substantially less simulator-derived energy. This provides direct evidence that the shortest path is not necessarily the least expensive path for a tracked vehicle.

The experiments also exposed the local and direction-dependent nature of the optimization problem. Reversing the high-to-low motion produced a low-to-high case with the same straight-line distance but no accepted update, and the three principal experiments accepted very different numbers of iterations. The physical objective depends on travel direction, slope, material interaction, and initialization; CHOMP improves the trajectory within the local cost landscape surrounding its initial solution and does not guarantee a global optimum. For this reason, the experiments were evaluated using trajectory geometry, knot displacement, path length, convergence history, physical energy, and tracking behaviour rather than through the final scalar objective alone.

The distributed implementation made this expensive procedure experimentally practical, although it did not

remove the underlying simulation cost. In the representative high-to-low run, 99 persistent workers distributed over 11 hosts completed 1120 gradient tasks and 1155 simulator calls. The mean gradient wall time was approximately 179.4 s and the mean line-search wall time was approximately 87.7 s. Exact simulator-backed CHOMP should therefore be regarded as a high-fidelity offline optimizer, reference method, or final refinement stage rather than as a lightweight online planner in its current form.

The learning branch evolved in response to limitations observed in the original short-trajectory dataset. V3 expanded the spatial scale and introduced broader procedural variation in terrain geometry, roughness, material fields, path shape, and local speed. V4 retained this structure while adding global duration variability, allowing the learning problem to represent both where the robot moves and how quickly the motion is executed.

Dataset	Worlds	Usable episodes	Trusted success	Trusted soft failure	Main coverage
V3	60	3000	2851	149	Geometry, roughness, materials, path shape and scale
V4	70	7000	6614	386	V3 coverage plus variable global duration

Trusted soft failures were retained because a physically valid failure is informative: it shows that a requested motion can lead to excessive deviation, poor goal reaching, or another undesirable outcome. The redesign also brought the trajectory scale closer to that of the planning experiments. The early dataset had a mean desired length of approximately 2.1 m and a maximum of approximately 3.9 m, whereas the V3 pilot reached approximately 18.0 m on average and 36.4 m at maximum.

Matched V3 and V4 model tasks were used to evaluate local mechanical-power prediction, direct trajectory-level scalar prediction, direct executed-trajectory prediction, and local forward transitions. The winning test results were as follows.

Model task	V3 winner	V3 test RMSE	V4 winner	V4 test RMSE
Local power	Transformer-small	2.661	ResMLP	3.065
Direct scalar	ResMLP	50.214	Transformer-small	39.670
Direct executed trajectory	GRU	0.261	GRU	0.242
Local forward transition	GRU	6.17×10^{-4}	GRU	4.50×10^{-4}

The numerical values are task-specific and should not be compared across rows because they refer to different targets and scales. In particular, the local-forward value is a one-step transition error and is not evidence of equivalent accuracy over a recursive trajectory rollout. Within these limits, V4 improved the best held-out RMSE for the direct-scalar, direct-trajectory, and one-step local-forward tasks, but not for local power. The additional timing variability therefore improved several model families without providing a uniform benefit across all tasks.

The decisive result emerged when the learned models were inserted into the planning loop. None of the evaluated V3 or V4 surrogate pipelines reproduced the metre-scale trajectory deformation of exact simulator-backed CHOMP. The V4 E3 direct-trajectory-plus-power pipeline reduced its predicted mechanical energy by only approximately 1.532 J and produced a mean knot displacement of approximately 0.015 78 m. The V4 E3 local-forward pipeline predicted a larger energy reduction of approximately 134.607 J, but its mean geometric

displacement was only approximately 1.5×10^{-4} m. The magnitude of the predicted scalar reduction was therefore not consistently related to a meaningful optimizer deformation.

This negative planning result does not invalidate the predictive models. Their held-out errors show that they learned useful aspects of the simulator distribution. It instead identifies a stricter requirement at the interface between learning and optimization. CHOMP estimates a descent direction from small differences between neighbouring perturbed trajectories; a model may predict absolute costs reasonably well while smoothing, attenuating, or reversing precisely those local differences. The evaluated surrogates can consequently be described as useful predictive approximations, but not yet as reliable replacements for the high-fidelity cost inside CHOMP.

Taken together, the thesis contributes a high-fidelity energy-aware optimization workflow, a validated finite-difference gradient method, a distributed persistent-simulation architecture, procedural V3 and V4 teacher datasets, and a matched evaluation of several surrogate-model families. Its most important diagnostic contribution is the demonstration that global prediction metrics must be complemented by local ranking, perturbation sensitivity, recursive rollout behaviour, meaningful trajectory deformation, and final high-fidelity replay when a learned model is intended for planning.

5.2 Limitations and Future Directions

The conclusions of this work are subject to several limitations. The terramechanics simulator is the highest-fidelity reference used in the thesis, but it remains a model and no final real-robot experiment was performed to measure the simulator-to-reality gap. CHOMP is a local optimizer, so the reported improvements apply to the studied initializations and do not establish global optimality. The completed exact experiments optimize spatial knots with fixed timing; although V4 adds duration variability to the learning dataset, joint optimization of trajectory geometry and total duration was not completed. In addition, the historical signed slip objective is not identical to non-negative physical slip dissipation, and the preliminary V4 local-forward comparison is based on one-step rather than full recursive-rollout error. Finally, the reduced-physics-plus-residual branch could not be compared with the same completeness because a compatible validated Reduced-ID baseline artifact was unavailable.

These limitations point to a focused continuation of the work rather than to a broad expansion in unrelated directions. The first priority is to align surrogate training with the information required by CHOMP. Future teacher data should include groups of neighbouring perturbations around actual planner states, and the loss should reward pairwise cost ordering, finite-difference gradient agreement, preservation of descent direction, and consistency across perturbation scales. Planner-in-the-loop data generation could then identify uncertain or inconsistent regions, query the high-fidelity simulator only where needed, and add those examples to the training distribution.

A second direction is to represent the problem at multiple spatial and temporal scales. Fine local features are needed for contact- and terrain-dependent effects, while segment-level features describe curvature and accumulated tracking error, and global features describe path length, duration, goal reaching, and large terrain structures. Combining these scales may be more suitable than expecting a purely local transition model or one global scalar predictor to preserve every feature relevant to optimization. Predictive uncertainty and out-of-distribution detection should also be included so that unreliable candidates can be sent back to the simulator instead of being exploited by the optimizer.

The third direction is to optimize geometry and timing jointly. V4 provides an initial data basis for studying duration as a planning variable. A duration-aware method would estimate both

$$\frac{\partial J}{\partial \boldsymbol{\xi}} \qquad \text{and} \qquad \frac{\partial J}{\partial T}, \tag{5.2}$$

subject to physically reasonable velocity and acceleration constraints. Because CHOMP remains local, this extension should be combined with diverse initializations or a global candidate generator. A realistic hybrid planner could use a learned model for broad candidate screening and uncertainty-aware exploration, followed by high-fidelity CHOMP refinement and a final simulator replay.

The final validation step should use the non-negative slip-dissipation formulation consistently and should compare mechanical energy, slip dissipation, tracking, and trajectory geometry under a common high-fidelity protocol. Selected trajectories should ultimately be executed on a real tracked platform to determine whether the predicted energetic advantages persist in the presence of sensing error, controller error, and unmodelled terrain

effects.

5.3 Final Perspective

This thesis began from the observation that geometric path length is an incomplete planning criterion for a tracked robot. The exact CHOMP experiments confirmed that detailed track–terrain interaction can create strong energetic preferences and can make a longer path physically less expensive than a straight one.

The work also quantified the price of this fidelity. A single gradient requires many complete simulator rollouts, making distributed execution, persistent workers, caching, and reliable checkpoint semantics essential. V3 and V4 established the data-generation infrastructure needed to study lower-cost approximations and demonstrated encouraging held-out performance across several prediction tasks.

The final planning experiments nevertheless establish a stricter definition of success. Predicting simulator outputs accurately on average is not equivalent to guiding an optimizer correctly. A planning surrogate must preserve the local structure of the physical objective closely enough to produce the correct descent direction, substantial trajectory deformation, and a solution that remains advantageous when replayed through the high-fidelity simulator.

The most defensible outcome is therefore not that high-fidelity simulation has already been replaced. Rather, this thesis establishes a complete physical reference pipeline, demonstrates meaningful energy-aware optimization, provides scalable teacher-data and evaluation infrastructure, and identifies local ranking fidelity as the central unresolved problem for surrogate-based CHOMP. These results provide a foundation for future hybrid planners in which learned models accelerate exploration while the high-fidelity simulator remains the final physical authority.

Bibliography

[1] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[2] Lester E. Dubins. On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents. *American Journal of Mathematics*, 79(3):497–516, 1957.

[3] Michele Focchi, Daniele Fontanelli, Davide Stocco, Riccardo Bussola, and Luigi Palopoli. Pseudo-kinematic trajectory control and planning of tracked vehicles. *Robotics and Autonomous Systems*, 197:105282, 2026.

[4] S. A. A. Moosavian and Arash Kalantari. Experimental slip estimation for exact kinematics modeling and control of a tracked mobile robot. In *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2008.

[5] Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7559–7566, 2018.

[6] Grady Williams, Andrew Aldrich, and Evangelos A. Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.

[7] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Information theoretic model predictive control: Theory and applications to autonomous driving. *arXiv preprint arXiv:1707.02342*, 2017.

[8] Genki Yamauchi, Keiji Nagatani, Takeshi Hashimoto, and Kenichi Fujino. Slip-compensated odometry for tracked vehicle on loose and weak slope. *ROBOMECH Journal*, 4:27, 2017.

[9] Matthew Zucker, Nathan Ratliff, Anca D. Dragan, Mihail Pivtoraiko, Matthew Klingensmith, Christopher M. Dellin, J. Andrew Bagnell, and Siddhartha S. Srinivasa. CHOMP: Covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research*, 32(9–10):1164–1193, 2013.